



Graduation Project



Thebes Academy
Faculty of Computer Science
Artificial Intelligence Department

EYEonic
(Gaze-Controlled Robotic Car for Hands-Free Interaction)

By

Omar Ahmad Abdelaziz

Youssef Reda Farouk

Rawda Hesham El Sayed

Habiba Magdy Eweda

Nada Ahmed Meky

Nadeen El Sayed Abdelazez

Sara Abdelkafy Hanfy

A graduation project is submitted to the **Computer Science** Department in partial fulfillment of the requirements for the degree of Bachelor of **Computer Science**, Thebes Higher Insitute for Computer and Management Sciences.

Supervisor

Dr. Khaled Abdelrhman



Cairo, Egypt.

2025/2026

Acknowledgements

We would like to express our deepest gratitude to **Dr. Khaled Abdelrhman** for his continuous support, guidance, and invaluable feedback throughout all stages of the development of this project. His academic expertise, constructive suggestions, and constant encouragement played a crucial role in shaping the direction and quality of this work.

We also extend our sincere thanks to **Thebes Academy**, Faculty of Computer Science, for providing a supportive academic environment, technical resources, and facilities that enabled us to successfully carry out this project. The knowledge and skills we gained during our years of study greatly contributed to the completion of this work.

Finally, we would like to acknowledge the dedication, cooperation, and collective effort of every member of our team. This project would not have been possible without the commitment, teamwork, and persistence shown throughout the planning, development, testing, and documentation phases

Abstract

This project presents **EYEonic**, a gaze-controlled robotic car designed to enable fully hands-free interaction using eye movements. The system integrates computer vision, artificial intelligence, and embedded systems to create an accessible robotic platform for individuals with limited hand mobility.

Using a standard webcam and MediaPipe Face Mesh, the system detects eye landmarks and analyzes gaze direction (left, right, forward) along with blink-based commands. These commands are processed in real time through a Python-based vision module and Wi-Fi UDP Communication

The ESP32 controls a 4-wheel robotic chassis using differential motor control, providing smooth motion and responsive navigation. Additional safety features—such as ultrasonic obstacle detection—ensure collision avoidance and stable operation.

EYEonic offers a low-cost, portable, and practical assistive solution, bridging the gap between vision-based tracking technologies and robotic control.

The project demonstrates the effectiveness of gaze-based interaction and highlights its potential in accessibility, rehabilitation, and smart mobility applications.

Keywords

Eye Tracking, Computer Vision, ESP32, Assistive Robotics, Human-Computer Interaction, MediaPipe , Machine Learning, Wi-Fi Communication, Human-Robot Interaction ,Obstacle Avoidance, Emergency Detection, Voice Interaction

Contents

ACKNOWLEDGEMENT.....	0
ABSTRACT.....	00
KEYWORDS.....	00
Chapter 1: Introduction.....	1
1.2 Problem Definition.....	2
1.3 Project Objectives	3
1.4 Motivation and Significance.....	4
1.5 Scope and Limitations	4
1.6 Report Organization	5
Chapter 2: Literature Review/ Related work.....	7
2.2 Comparison between existing solutions	7
2.3 Strengths and weaknesses of previous work.....	8
2.3 summary and how your project builds upon or differs from them	9
Chapter 3: System Analysis /Methodology.....	11
3.1.1 Functional Requirement	11
3.1.2 Non-functional requirements.....	14
3.1.3 User Requirement	15
3.1.4 System Constrains.....	16
3.2 System Feasibility.....	16
3.2.1 Technical feasibility	16
3.2.2 Operational Feasibility	17
3.2.3 Economic Feasibility	17
3.3 Research or Development Methodology	17

Chapter 1: Introduction

1.1 Background and Context

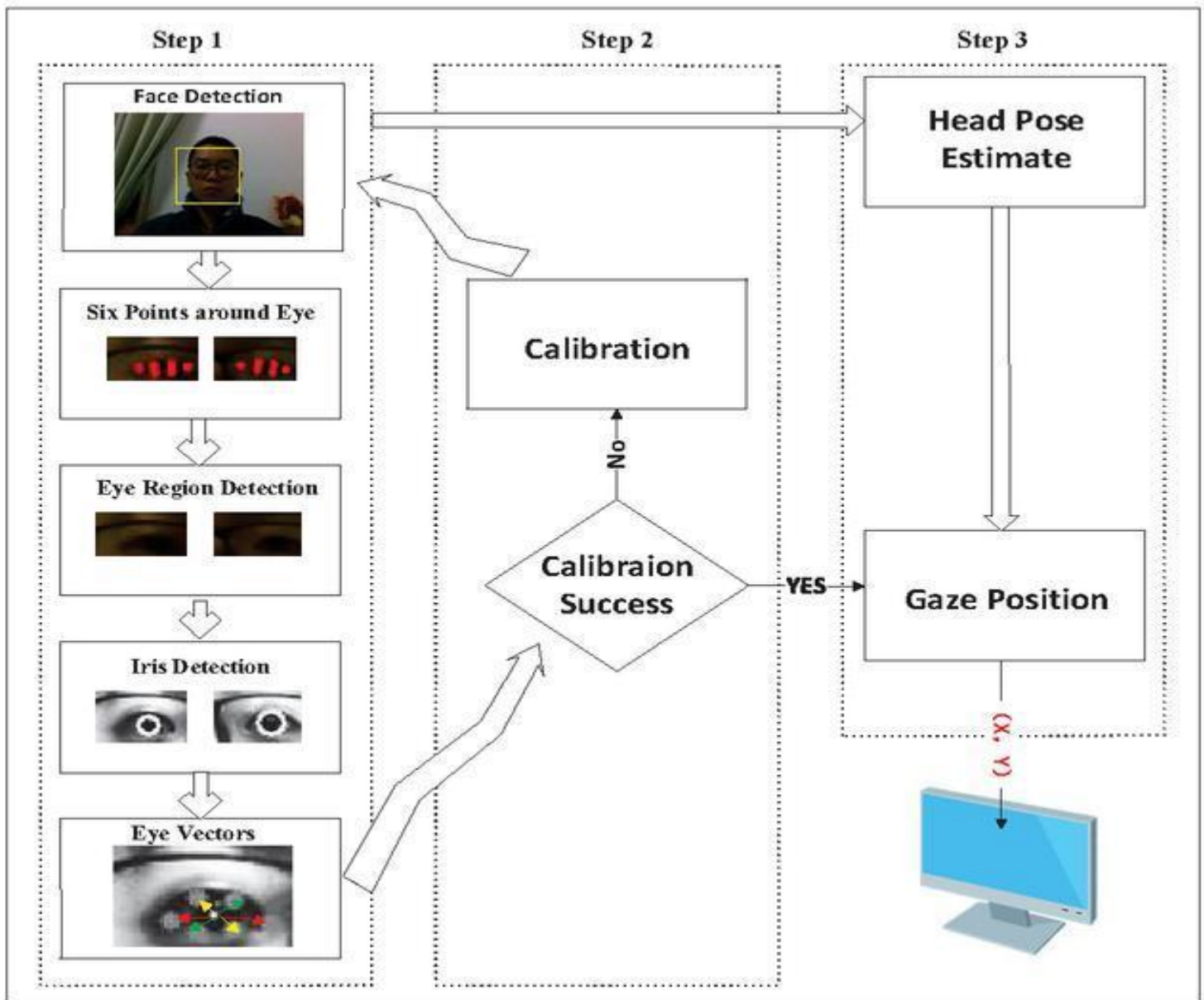
in recent years, **Human–Computer Interaction (HCI)** has expanded beyond traditional input methods such as keyboards, mice, and touch-based interfaces toward more **natural, intuitive, and accessible interaction modalities**. This shift aims to reduce the physical and cognitive effort required from users while enhancing usability and inclusiveness. Among these emerging technologies, **eye-tracking** has gained significant attention as a non-invasive and effective technique for capturing and interpreting human gaze patterns to control both digital and physical systems in real time.

Eye-tracking technology is particularly impactful for individuals with **limited mobility or motor impairments**, as it enables hands-free interaction with machines and devices. By relying on eye movements rather than physical input, such systems can significantly improve user independence, accessibility, and overall quality of life. Beyond assistive applications, gaze-based interaction has also been utilized in gaming, virtual reality, human attention analysis, and intelligent user interfaces.

At the same time, **robotics and embedded systems** have witnessed rapid advancements in terms of processing power, energy efficiency, and miniaturization. Modern microcontrollers and sensors now allow the development of compact, low-cost, and intelligent robotic platforms capable of responding to external inputs with high precision and reliability. The integration of **computer vision, machine learning, and embedded control systems** has further enabled robots to perceive their environment, interpret user intent, and perform actions autonomously or semi-autonomously.

By combining these technological domains, this project aims to bridge the gap between human intent and robotic motion. The proposed system integrates eye-tracking through computer vision with embedded motor control to create an **assistive robotic platform that is driven entirely by eye movement**. This approach demonstrates how gaze-based interaction can be effectively translated into real-world robotic control, offering a practical, affordable, and scalable solution for hands-free human–robot interaction.

Eye Tracking Technology Concept



1.1.1 Human-Robot Interaction

Human-Robot Interaction (HRI) is a rapidly growing field that focuses on developing intuitive communication methods between humans and robotic systems. Traditional interaction methods such as keyboards, joysticks, and touch interfaces may not be suitable for individuals with severe motor impairments. Therefore, alternative interaction modalities such as eye tracking, voice commands, and intelligent assistive technologies have gained significant attention in recent years.

The integration of gaze-based control with voice interaction provides a more natural and accessible user experience while improving the overall efficiency and usability of assistive robotic systems.

Human-Robot Interaction Concept



1.2 Problem Definition

Many individuals with motor disabilities experience significant difficulties when interacting with physical environments due to limited or impaired hand mobility. Daily tasks that rely on manual input devices—such as joysticks, buttons, touchscreens, or mobile applications—can become extremely challenging or even impossible for such users. As a result, conventional mobility and control systems, including wheelchairs, robotic platforms, and remote-controlled devices, often fail to meet the accessibility needs of all users.

Despite recent technological advancements, there remains a noticeable lack of **accessible, intuitive, and cost-effective gaze-controlled robotic platforms** that enable individuals to interact with physical systems without relying on hand-based input. Existing solutions are frequently expensive, complex, or dependent on specialized hardware, which limits their widespread adoption and practical use.

Therefore, this project addresses the following research problem:

How can eye movements be accurately and efficiently translated into real-time control commands for a mobile robotic system in order to achieve fully hands-free navigation?

The proposed solution seeks to overcome current limitations by leveraging computer vision and embedded systems to create a reliable, affordable, and user-friendly gaze-controlled robotic platform.

Assistive Robotics for Disabled Users



1.2.1 Safety Challenges

Safety remains one of the major challenges in assistive robotic systems. Users may encounter unexpected obstacles, communication failures, or emergency situations that could compromise safe operation. Therefore, modern assistive robots require integrated safety mechanisms such as obstacle detection, emergency stopping procedures, visual indicators, and auditory warnings to ensure reliable operation in real-world environments.

1.3 Project Objectives

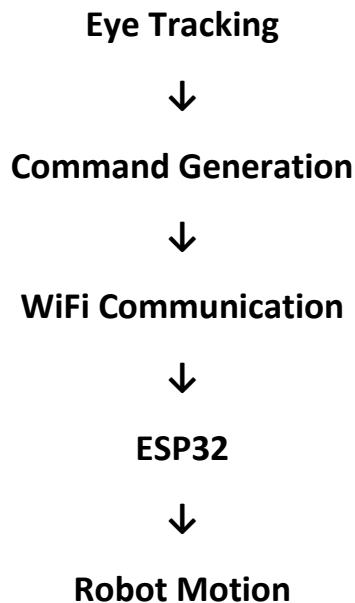
The main objective of this project is to **design and develop a gaze-controlled robotic car** that can respond to eye movements in real time by integrating **computer vision techniques** with **embedded control systems**. The system aims to enable fully hands-free navigation, providing an intuitive and accessible interaction method, particularly for users with limited hand mobility.

To achieve this main goal, the project focuses on the following specific objectives:

1. **To design and implement an eye-tracking module** using computer vision techniques, including **OpenCV and MediaPipe**, in order to accurately detect facial landmarks and eye regions from a standard webcam.
2. **To develop a gaze interpretation system** capable of identifying gaze directions (left, right, and forward) as well as blink-based actions, ensuring reliable recognition of user intent under real-time conditions.
3. **To establish a robust and reliable communication interface** between the vision-processing module developed in Python and the **ESP32 microcontroller**, enabling efficient transmission of gaze-based control commands with minimal latency.
4. **To design and construct a four-wheel robotic platform** that translates gaze commands into smooth and stable motion using differential motor control techniques, ensuring precise maneuverability and user comfort.
5. **To integrate safety mechanisms** such as obstacle detection using ultrasonic sensors, allowing the robotic system to automatically prevent collisions and enhance operational reliability.

6. **To evaluate the overall system performance** in terms of accuracy, response time (latency), and usability, in order to assess the effectiveness and practicality of the proposed solution in real-world scenarios.
7. Develop a voice interaction module that enables users to control the robotic vehicle using speech commands.
8. Implement real-time obstacle avoidance using dual ultrasonic sensors.
9. Integrate emergency detection mechanisms capable of identifying abnormal situations such as prolonged eye closure.
10. Provide auditory and visual feedback using buzzer and LED indicators to enhance user awareness and safety.

Project Objective Flow



1.4 Motivation and Significance

- This project is motivated by the growing need to support individuals who are unable to use their hands to control devices due to **physical disabilities, injuries, neurological conditions, or temporary motor limitations**. For many of these individuals, interacting with traditional input devices such as keyboards, joysticks, or touchscreens can be extremely challenging, which limits their ability to independently operate assistive or mobility systems.
- A **low-cost, gaze-controlled mobility platform** has the potential to significantly enhance independence, accessibility, and quality of life by providing an intuitive and hands-free method of interaction. By relying on natural eye movements, users can control robotic systems with minimal physical effort, reducing dependency on caregivers and increasing personal autonomy.

- The significance of this project can be summarized as follows:
- **Introducing an affordable and accessible alternative** to expensive commercial eye-tracking devices, making assistive technology more widely available to a larger range of users.
- **Demonstrating the practical integration of computer vision and embedded systems** to develop real-world assistive robotic solutions that are both effective and scalable.
- **Encouraging innovation in hands-free interaction technologies**, particularly in the fields of assistive robotics, smart mobility systems, and human–robot interaction.
- **Providing an educational and research contribution**, showcasing how multidisciplinary engineering concepts can be combined to solve real societal challenges.
- Overall, this project highlights the potential of gaze-based control systems as a meaningful step toward more inclusive and intelligent assistive technologies.

1.4.1 Social Impact

The proposed system aims to improve the quality of life for individuals with physical disabilities by providing an affordable, intelligent, and easy-to-use mobility assistance solution. The project combines modern artificial intelligence techniques with embedded systems to create a practical assistive technology platform that can be further extended for healthcare and rehabilitation applications.

1.5 Scope and Limitations

Scope

The scope of this project defines the boundaries and functionalities of the proposed system. The project focuses on the design and implementation of a **gaze-controlled robotic car** that operates using computer vision and embedded systems technologies. Specifically, the scope includes:

- **Eye tracking using a standard webcam** combined with computer vision techniques to capture and analyze eye movements without the need for specialized or expensive hardware.
- **Real-time gaze direction estimation**, allowing the system to detect user intent by classifying gaze directions such as left, right, and forward with minimal delay.

- **Robotic car movement control** using an **ESP32 microcontroller** and motor drivers, enabling smooth and responsive translation of gaze commands into physical motion.
- **Obstacle detection and collision avoidance** through the integration of ultrasonic sensors, enhancing system safety during operation.
- **Smooth and natural robotic navigation**, achieved through differential motor control and command filtering to ensure user comfort and stable movement.
- **Voice Commands**
- **Emergency Alert System**
- **Dual Ultrasonic Sensors**

Overall, the project aims to demonstrate a practical and low-cost assistive robotic system capable of hands-free control in controlled indoor environments.

Limitations

Despite the advantages of the proposed system, several limitations are acknowledged:

- The system's performance **depends heavily on lighting conditions**, as insufficient or uneven lighting can negatively affect the accuracy of eye tracking.
- **Webcam-based eye tracking** may offer lower precision compared to dedicated infrared eye-tracking hardware, particularly in cases involving rapid head movement or partial occlusion.
- The system is **not designed for outdoor usage or high-speed navigation**, as environmental variations and safety constraints are beyond the intended scope of this project.
- The user must **remain within the camera's field of view** for reliable gaze detection, which restricts the range of head movement during operation.

1.6 Report Organization

The report is organized into eight chapters, each addressing a specific aspect of the proposed system, as follows:

- **Chapter 1: Introduction** provides an overview of the project background, problem statement, objectives, motivation, significance, scope, and limitations.
- **Chapter 2: Literature Review / Related Work** reviews existing eye-tracking technologies, gaze-controlled systems, and related robotic solutions, highlighting their strengths, weaknesses, and limitations.

- **Chapter 3: System Analysis / Methodology** explains the overall system methodology, requirements analysis, data flow, and the conceptual approach used to translate eye movements into robotic control commands.
- **Chapter 4: System Design / Proposed Solution** presents the detailed system architecture, including the vision module, embedded control module, hardware components, and the interaction between different system layers.
- **Chapter 5: Implementation** describes the practical implementation of the proposed system, covering computer vision algorithms, communication protocols, embedded programming, and hardware integration.
- **Chapter 6: Testing and Evaluation** outlines the testing procedures, test scenarios, and evaluation metrics used to assess system performance, accuracy, and reliability.
- **Chapter 7: Results and Discussion** analyzes the experimental results, discusses system performance, limitations, and compares the outcomes with the project objectives and existing solutions.
- **Chapter 8: Conclusion and Future Work** summarizes the overall project achievements and suggests potential improvements and future enhancements for further development.

Chapter 2: Literature Review / Related Work

2.1 Overview of Existing Systems

Existing gaze-controlled systems can generally be classified into two main categories based on the type of technology used for eye-tracking: **hardware-based eye tracking systems** and **software-based eye tracking systems**. Each category has its own characteristics, advantages, and limitations.

2.1.1 Voice-Controlled Assistive Systems

Voice-controlled assistive systems have become increasingly popular due to their ability to provide natural and hands-free interaction. Such systems typically utilize speech recognition algorithms to convert spoken commands into executable actions. Examples include smart wheelchairs, home automation systems, and personal robotic assistants.

Although voice-based interaction provides convenience, it may suffer from environmental noise and speech recognition limitations. Therefore, combining voice control with gaze-based interaction can significantly improve system reliability and accessibility.

Feature	Existing Systems	EYEonic
Eye Tracking	Yes	Yes
Voice Commands	Limited	Yes
Obstacle Detection	Limited	Dual Sensors
Emergency Detection	No	Yes
WiFi Control	Rare	Yes
Audio Feedback	Limited	Yes
Emergency Alerts	No	Yes

2.1.2 Hardware-Based Eye Trackers

Hardware-based eye tracking systems rely on **specialized equipment** to achieve highly accurate gaze detection. These systems typically use **infrared (IR) light sources**, high-speed cameras, and proprietary algorithms to precisely track eye movements and pupil positions. The infrared illumination allows for robust pupil detection under various lighting conditions and reduces noise caused by reflections or shadows.

Well-known examples of such systems include:

Commercial Eye Tracking Devices

Tobii Eye Tracker



EyeLink , Pupil Labs



SmartGlass

These devices are widely used in medical research, psychology experiments, usability studies, and high-end assistive technologies. They offer **very high accuracy**, **low latency**, and **stable performance**, even during rapid eye movements. However, despite their technical advantages, hardware-based eye trackers are often **expensive**, require **specialized calibration procedures**,

and depend on dedicated hardware components. This significantly limits their accessibility and makes them impractical for low-cost or consumer-level assistive robotic applications.

2.1.3 Software-Based Eye Tracking

Software-based eye tracking systems utilize **standard webcams** combined with **computer vision algorithms** to estimate gaze direction without the need for specialized hardware. These approaches focus on detecting facial features and eye regions directly from video frames captured by conventional cameras.

Common techniques used in software-based eye tracking include:

- **Haar cascades** for detecting faces and eye regions.
- **Hough Transform methods** for pupil segmentation and circular shape detection.
- **Facial landmark detection**, such as **MediaPipe Face Mesh**, to identify key eye and facial points.
- **Thresholding and contour detection** techniques to isolate the pupil from surrounding eye features.

These solutions are generally **more affordable**, portable, and easier to deploy compared to hardware-based systems. They are suitable for real-time applications using consumer-grade devices and open-source libraries. However, their performance can be **sensitive to environmental conditions**, particularly variations in lighting, camera quality, and head movement. As a result, software-based eye tracking systems may exhibit reduced accuracy and stability when compared to infrared-based solutions.

2.1.4 Summary

In summary, hardware-based eye tracking systems offer superior accuracy and robustness but suffer from high cost and limited accessibility, while software-based eye tracking solutions provide a **cost-effective and flexible alternative** at the expense of reduced precision under certain conditions. This trade-off highlights the need for efficient software-based approaches that balance performance, affordability, and usability—an objective that the proposed EYEonic system aims to address.

2.2 Existing Robotic Solutions

Several research studies and experimental systems have explored the integration of **eye-tracking technology with robotic control** in order to enable hands-free interaction and navigation. These solutions aim to translate user gaze or eye-related actions into robotic commands, particularly in assistive and human–robot interaction applications.

Examples of existing gaze-controlled robotic systems include:

- **Gaze-controlled electric wheelchairs**, where eye movements are used to control the direction and speed of the wheelchair for users with severe motor impairments.
- **Eye-tracking–based pointer control systems**, in which gaze is used to select targets or issue commands to robotic manipulators or mobile robots.
- **Mobile robotic platforms navigated using blinking actions or gaze direction**, where simple eye gestures trigger predefined robot movements such as turning, stopping, or moving forward.

While these systems demonstrate the feasibility of eye-based robotic control, many of them suffer from notable limitations. A significant number of existing solutions **depend on expensive and specialized eye-tracking hardware**, which restricts their availability and practical deployment. Additionally, several systems are designed to operate only in **controlled or limited environments**, such as laboratories or fixed indoor setups, reducing their adaptability to real-world scenarios.

Furthermore, many gaze-controlled robotic systems exhibit **limited or abrupt motion control**, resulting in non-smooth or unstable navigation that can negatively affect user comfort and system reliability. In addition, **obstacle detection and collision avoidance mechanisms** are often absent or minimally implemented, which raises safety concerns during operation.

These limitations highlight the gap between experimental gaze-controlled robotics and practical, user-friendly assistive systems. Addressing issues such as cost, smooth motion control, environmental adaptability, and integrated safety features remains a key challenge in this research area and serves as a motivation for the proposed EYEonic system.

Example of Gaze-Controlled Wheelchair



2.3 Comparison to Proposed System

This section presents a comparison between existing gaze-controlled robotic systems and the proposed EYEonic system. The comparison focuses on key aspects such as cost, technology, mobility, safety, accessibility, and motion smoothness in order to highlight the advantages and contributions of the proposed solution.

Many existing systems prioritize accuracy by relying on specialized hardware and proprietary algorithms; however, this often results in high cost and limited accessibility. In contrast, the EYEonic system is designed to balance performance and affordability by leveraging computer vision techniques and open-source tools. This approach enables broader adoption while maintaining reliable functionality.

Comparison Table

Feature	Existing Systems	Proposed EYEonic System
Cost	High cost due to specialized eye-tracking hardware	Low cost using a standard webcam and open-source libraries
Technology	Infrared sensors and proprietary algorithms	Computer vision techniques using OpenCV and MediaPipe

Feature	Existing Systems	Proposed EYEonic System
Mobility	Limited or basic robotic control	Full robotic navigation with multi-directional movement
Safety	Not always available or minimally implemented	Integrated obstacle avoidance using ultrasonic sensors
Accessibility	Requires dedicated and specialized equipment	Operates using common laptops and standard cameras
Motion Smoothness	Highly dependent on hardware quality	Optimized motor control using PWM and smooth curve motion

Discussion

As shown in the table, the proposed EYEonic system offers several improvements over existing gaze-controlled robotic solutions. By eliminating the need for expensive infrared-based eye trackers, EYEonic significantly reduces system cost while improving accessibility. The use of computer vision and machine-learning-based facial landmark detection allows the system to operate using widely available hardware, making it suitable for educational, research, and assistive applications.

In addition, EYEonic emphasizes smooth and natural robotic motion through optimized PWM motor control and differential steering techniques, addressing a common limitation observed in many previous systems. The inclusion of obstacle detection and safety mechanisms further enhances system reliability and user confidence during operation.

Overall, this comparison demonstrates that the proposed EYEonic system effectively bridges the gap between high-cost, hardware-dependent solutions and low-cost but limited software-based approaches, providing a practical and scalable gaze-controlled robotic platform.

2.4 Strengths and Weaknesses of Previous Works

This section highlights the main strengths and weaknesses observed in previous gaze-controlled and eye-tracking-based robotic systems, based on findings reported in related research and existing implementations.

Strengths

Several previous works demonstrate notable strengths, particularly those relying on hardware-based eye-tracking technologies. These strengths include:

- **High accuracy achieved through infrared (IR) sensors, which enable precise pupil detection under varying lighting conditions and reduce noise caused by reflections or shadows.**
 - **Reliable and stable pupil tracking, especially in systems that use high-speed cameras and dedicated eye-tracking hardware.**
 - **Strong commercial and software support, such as proprietary development kits (e.g., Tobii SDK), which provide optimized tools, documentation, and technical assistance.**
 - **Low system latency in hardware-based trackers, allowing real-time gaze detection with minimal delay, which is critical for responsive interaction.**
-

Weaknesses

Despite these advantages, previous systems exhibit several limitations that restrict their practical usability and accessibility:

- **Very high cost, which significantly limits accessibility and prevents widespread adoption, particularly in educational and low-budget assistive applications.**
 - **Limited portability, as many systems require bulky setups, fixed screens, or dedicated mounting positions, making them unsuitable for mobile or flexible environments.**
 - **Instability in software-only eye-tracking approaches, especially under poor or uneven lighting conditions, leading to reduced accuracy and unreliable gaze detection.**
 - **Lack of full embedded robotic integration in many solutions, where gaze detection is often demonstrated without complete real-time control of a physical robotic platform.**
 - **Limited system completeness, as few existing works successfully combine gaze-based control, obstacle detection, safety mechanisms, and smooth robotic motion into a single integrated platform.**
-

Summary

These strengths and weaknesses reveal a clear research gap in the development of gaze-controlled robotic systems that are both accurate and accessible. While previous works excel in precision and responsiveness, they often fall short in terms of cost, portability, safety integration, and real-world applicability—challenges that the proposed EYEonic system aims to address.

2.5 How EYEonic Builds Upon Previous Work

The proposed **EYEonic system** builds upon previous research in gaze-controlled systems and assistive robotics by addressing several of the key limitations identified in earlier works. While existing approaches have demonstrated the feasibility of eye-based interaction, EYEonic enhances these concepts by focusing on **affordability, practicality, system integration, and real-world applicability**.

First, EYEonic adopts a **low-cost implementation strategy** by relying exclusively on open-source software tools and a standard webcam. Unlike many previous systems that require expensive infrared-based eye-tracking hardware, the proposed solution significantly reduces overall system cost while maintaining acceptable accuracy and performance.

Second, the system utilizes **advanced machine-learning-based facial landmark detection**, such as MediaPipe Face Mesh, to achieve **real-time gaze estimation**. This approach improves robustness compared to traditional feature-based methods and allows accurate detection of eye movement without the need for specialized sensors.

Third, EYEonic goes beyond cursor or pointer-based interaction by enabling **direct control of a physical four-wheel robotic platform**. This transformation from screen-based interaction to real-world robotic navigation represents a substantial step forward in practical assistive robotics.

In addition, the proposed system emphasizes **smooth and natural robotic navigation** through the use of differential motor control and PWM blending techniques. This ensures gradual turns and stable motion, addressing the abrupt or unstable movements observed in many existing systems.

Safety is also a core component of the EYEonic design. The integration of **ultrasonic sensors for obstacle detection and collision avoidance** enhances system reliability and user confidence, a feature that is often missing or underdeveloped in previous works.

Finally, EYEonic is designed with a **modular system architecture**, allowing future extensions such as head movement tracking, voice control, or additional sensors without requiring major system redesign. This flexibility makes the system suitable for further research and long-term development.

In summary, EYEonic effectively bridges the gap between **software-based eye-tracking solutions** and **physical-world robotic interaction**, offering a comprehensive, scalable, and accessible gaze-controlled robotic platform that improves upon the limitations of prior approaches

2.6 Research Gap

Most existing assistive robotic systems focus on a single interaction modality, such as eye tracking or voice control. Furthermore, many systems lack integrated safety mechanisms and emergency response features. The proposed EYEonic system addresses these limitations by combining gaze-based control, voice interaction, obstacle avoidance, emergency detection, and wireless communication into a unified intelligent platform. The proposed system also integrates dual-layer obstacle avoidance and emergency alert mechanisms, which are rarely combined within a single low-cost assistive robotic platform.

Chapter 3 : System Analysis & Design

3.1 Requirement Analysis

Requirement analysis is a critical phase in system development, as it defines the expected functionality, performance, and constraints of the proposed system. This section outlines the **functional, non-functional, user, and system requirements** of the EYEonic project to ensure clarity, feasibility, and alignment with project objectives.

3.1.1 Functional Requirements

The functional requirements describe what the system is expected to do in terms of features and operations. The proposed system must fulfill the following functional requirements:

The system must enable hands-free control of a four-wheel robotic car using eye movements as the primary input method.

The system must detect eye movements in real time using a standard camera and computer vision software.

The detected gaze directions must be accurately converted into corresponding motion commands, as follows:

Forward gaze → Move the robotic car forward

Left gaze → Turn the robotic car left

Right gaze → Turn the robotic car right

Blink detection → Stop the robotic car immediately

The system does not support backward movement, as the chassis is not equipped with a rear-facing obstacle sensor. Restricting motion to forward, left, right, and stop ensures that every directional movement can be verified as safe before it is executed.

Before executing any directional movement command, the system must perform a "Scan-before-Move" procedure: the scanning ultrasonic sensor is rotated via a servo motor toward the intended direction of motion, the distance to the nearest obstacle in that direction is measured, and the movement command is executed only if the measured distance exceeds a predefined safety threshold; otherwise, the car remains stationary and an alert is activated.

The system must control two DC motors configured in an axle-based drive mechanism to efficiently drive all four wheels.

The system must support wireless communication between the vision-processing unit (PC) and the robotic car using Bluetooth or Wi-Fi, enabling flexible and untethered operation.

Voice Interaction Requirements

- The system shall support voice-based commands for controlling robot movement.
- The system shall provide auditory feedback to inform users about system status and received commands.
- The system shall process voice commands in real-time with minimal latency.

3.1.2 Non-Functional Requirements

Safety:

The system incorporates a multi-level safety mechanism to ensure reliable and secure operation:

Eye Inactivity Monitoring: If no command is received from the eye-tracking module for more than 15 seconds, the robotic car automatically stops and enters a safe-mode state until a new command is received.

Sensor Fault Detection: If either ultrasonic sensor continuously returns invalid (zero) readings for more than 3 seconds, the system flags a sensor fault, activates a distinct fault-alert pattern on the LED and buzzer, and restricts the directional movements that depend on the faulty sensor.

Remote Emergency Notification: Using an integrated Wi-Fi module, the system monitors the user's eye-closure state via the computer vision module. If the eyes remain closed for more than 15 seconds, an emergency alert is automatically sent to a caregiver via the Telegram messaging platform, with the alert repeating periodically as long as the condition persists.

Emergency Requirements

- The system shall detect prolonged eye closure and activate emergency mode.
- The system shall immediately stop the robotic vehicle during emergency situations.
- The system shall generate visual and auditory alerts when emergency conditions are detected.

3.1.3 User Requirements

User requirements focus on usability and accessibility, particularly for the target user group of individuals with limited mobility. The system must satisfy the following user-centered requirements:

- Users should be able to **control the robotic car without any physical hand or body movement**, relying solely on eye movements.
- The user interface must be **simple, intuitive, and easy to use**, requiring minimal configuration and calibration before operation.
- The system should be **accessible to individuals with severe mobility impairments**, ensuring that no specialized technical knowledge or complex setup procedures are required.

3.1.4 System Constraints

System constraints define the limitations under which the system must operate. These constraints influence design decisions and implementation choices:

- The system is constrained to using **only two DC motors** to drive all four wheels through an axle-based drive mechanism.
- The accuracy and reliability of gaze detection are **limited by the camera resolution**, frame rate, and environmental lighting conditions.
- **Budget constraints** restrict the use of high-end sensors, infrared eye trackers, or advanced motor systems, necessitating the use of cost-effective components.
- The system must be **compatible with commonly available laptops or personal computers**, without requiring specialized or proprietary hardware.

3.1.5 Communication Requirements

The system shall support real-time wireless communication between the AI processing unit and the ESP32 controller using Wi-Fi UDP protocol.

The communication latency shall remain sufficiently low to ensure responsive robot navigation.

The system shall continue operating reliably within the coverage area of the ESP32 wireless network.

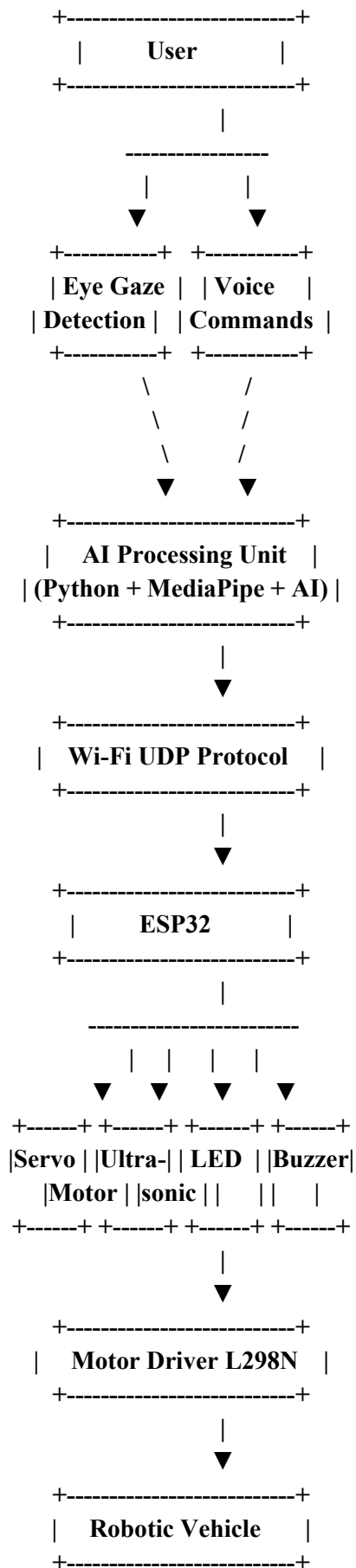
3.1.6 Hardware Safety Requirements

The system shall continuously monitor two ultrasonic sensors.

The front ultrasonic sensor shall monitor obstacles directly ahead.

The servo-mounted ultrasonic sensor shall scan the intended direction before movement.

Any detected obstacle within the safety threshold shall immediately stop the robotic vehicle.



Flowchart Diagram

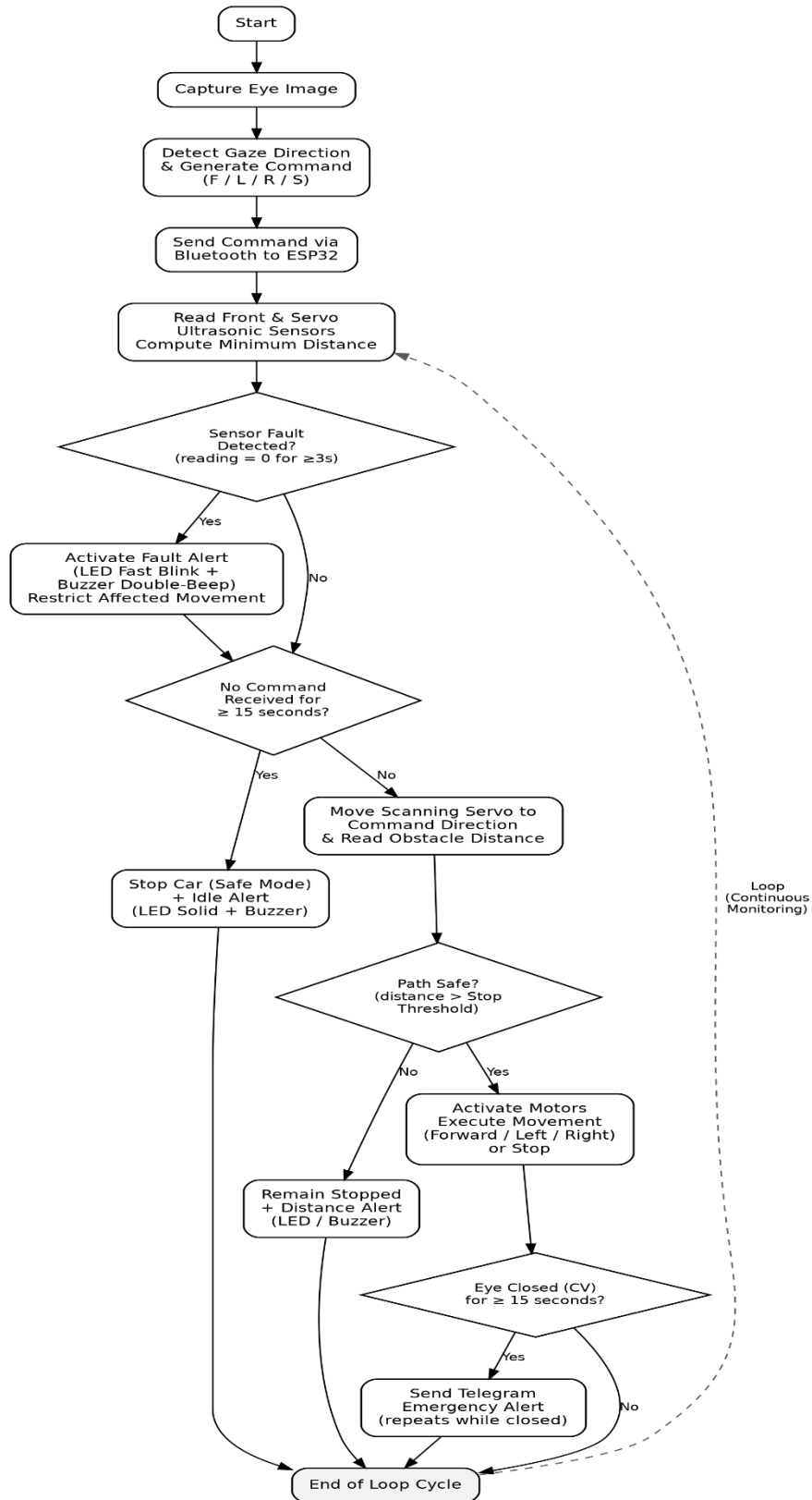


Figure 1: Flowchart showing the process of converting eye movement into robotic car motor commands

Use Case Diagram

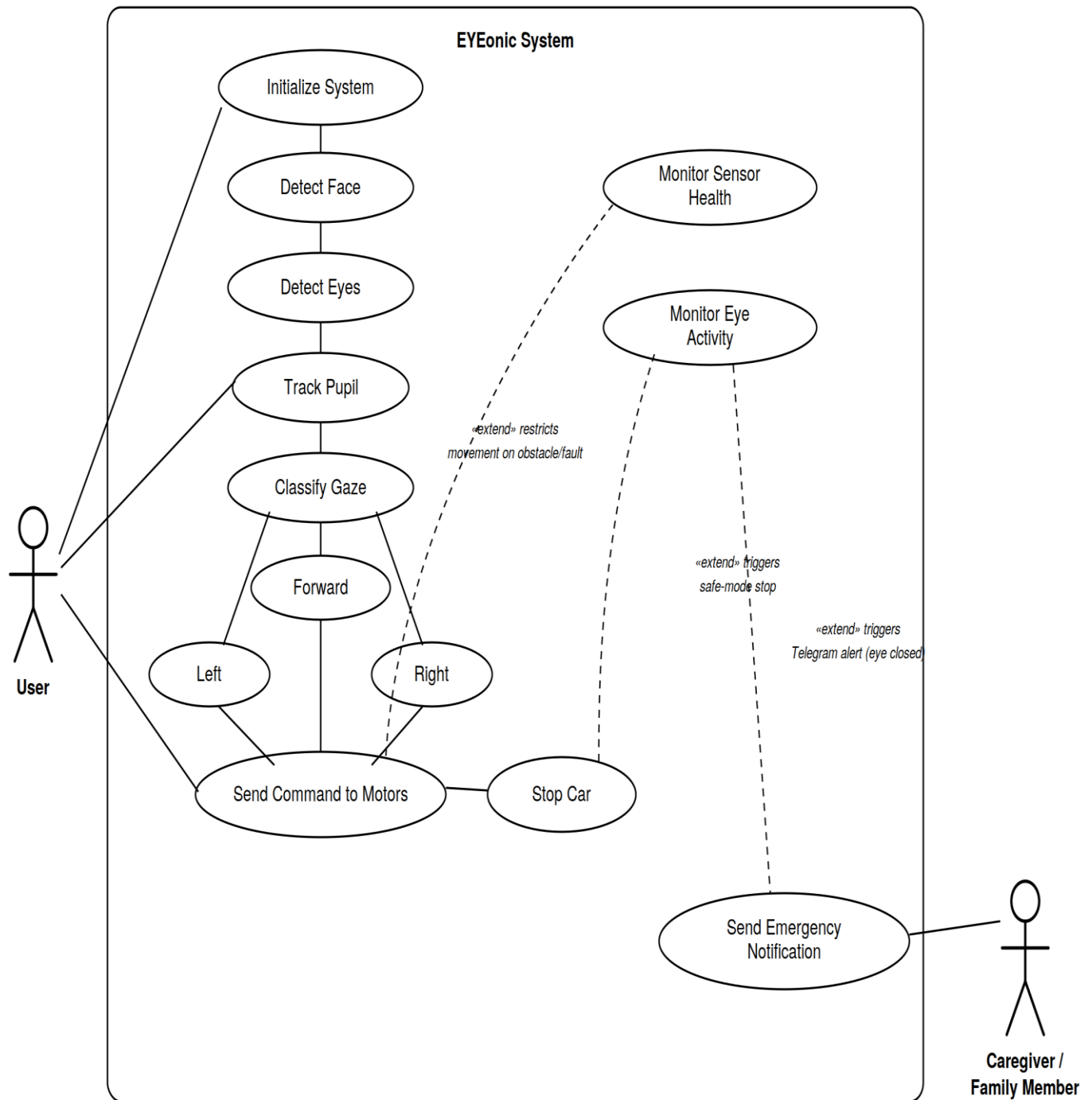


Figure 2: Use Case Diagram illustrating the interaction between the user and the eye-controlled robotic car system functions.

3.2 System Feasibility

System feasibility analysis is conducted to evaluate the practicality and viability of implementing the proposed EYEonic system. This analysis covers **technical, economic, and operational feasibility** to ensure that the project can be successfully developed within the available resources, time, and constraints.

3.2.1 Technical Feasibility

The proposed system is technically feasible, as all required hardware and software components are readily available and compatible with each other. The system relies on commonly used technologies that are well-documented and widely supported.

- All essential hardware components, including a **standard camera, ESP32 microcontroller, DC motors, and motor driver**, are commercially available and easy to integrate into a single robotic platform.
- Eye-tracking functionality can be implemented using **open-source computer vision libraries** such as **MediaPipe Face Mesh, OpenCV, NumPy, WiFi UDP**, which provide reliable tools for face detection, eye localization, and feature extraction.
- Wireless communication technologies such as **Bluetooth or Wi-Fi** are widely supported by the ESP32 microcontroller and can be programmed efficiently to enable real-time data transmission between the PC and the robotic car.
- The required programming environments for both the vision module (Python) and the embedded module (Arduino/ESP32 framework) are mature, stable, and well-supported by online documentation and community resources.

Based on these factors, the technical implementation of the EYEonic system is achievable without the need for specialized or proprietary technologies.

3.2.2 Economic Feasibility

The project is economically feasible and suitable for development under a limited academic budget. Cost reduction is a key design consideration in the proposed system.

- Using **two DC motors to drive four wheels** through an axle-based mechanism significantly reduces hardware cost compared to a four-motor configuration.
- eye-tracking solutions and development tools.
- The use of affordable, off-the-shelf components ensures that the overall system cost remains low while still meeting functional and performance requirements.
- The project does not require expensive sensors, infrared eye trackers, or high-end processing units, making it financially accessible for educational and assistive applications.

Overall, the system can be developed within a reasonable budget without compromising essential functionality.

3.2.3 Operational Feasibility

Operational feasibility evaluates how effectively the system can be used in real-world scenarios by the intended users.

- The system is designed to be **user-friendly and intuitive**, allowing individuals with physical or motor limitations to operate the robotic car without extensive training.
- Hands-free control through eye movements minimizes physical effort and enhances usability for users with severe mobility impairments.
- The **four-wheel chassis design** provides stability and balance, ensuring safe operation during movement and turns.
- Real-time eye-tracking and command execution allow for **efficient and reliable control**, improving user confidence and overall system effectiveness.

These factors indicate that the proposed system is operationally feasible and suitable for its intended purpose.

3.3 Research / Development Methodology

This project follows an **application-based development approach** guided by the **Software Development Life Cycle (SDLC)**. Given the clear definition of system requirements and objectives, the **Waterfall Model** is selected as an appropriate methodology for system development.

Safety Layer

A dedicated safety layer has been integrated into the system architecture. This layer continuously monitors ultrasonic sensor readings, emergency eye closure events, and communication status. In the presence of any unsafe condition, the system immediately activates emergency procedures, including vehicle stopping and user notification through visual and auditory alerts.

3.3.1 Application-Based Approach (SDLC – Waterfall Model)

The development process consists of the following sequential phases:

1. Requirement Analysis

In this phase, all functional, non-functional, user requirements, and system constraints are identified and documented to establish a clear foundation for development.

2. System Design

The system architecture and component layout are defined, including:

- **Hardware design:** four-wheel robotic chassis, two DC motors, ESP32 microcontroller, camera, motor driver, power supply, and wireless communication module.
- **Software design:** vision module for eye detection, gaze classification, command generation, and embedded motor control logic.

3. Implementation

During this phase:

- Eye-tracking software is developed using **Python with OpenCV and Dlib** to detect facial features and estimate gaze direction.
- The vision module is interfaced with the **ESP32 microcontroller**, which processes received commands and controls the motors accordingly.

4. Testing

The system undergoes multiple testing stages, including:

- **Functional testing** to verify that all gaze commands (forward, left, right, stop) are correctly executed.
- **Performance testing** to evaluate response time, system stability, and real-time behavior.

5. Deployment and Maintenance

After successful testing:

- The system is deployed for end-user operation.
- Maintenance activities include software updates, parameter tuning, and troubleshooting of hardware-related issues.

3.3.2 Eye Tracking Algorithm

The eye-tracking algorithm forms the core of the vision-processing module and consists of several sequential steps:

- **MediaPipe Face Mesh** is used to extract facial and eye landmarks. The iris position is then analyzed to estimate gaze direction (left, right, and forward). Blink detection is performed using Eye Aspect Ratio (EAR) calculations, allowing the system to identify stop and emergency commands..
- **Eye detection and tracking** involves extracting the eye regions from the detected face and continuously tracking the pupil position across video frames.
- **Gaze classification** determines the user's gaze direction (left, right, forward) and identifies blinking actions based on eye movement patterns.
- **Command generation** translates the classified gaze direction into corresponding motor control signals, which are then transmitted to the embedded system for execution.

This algorithmic pipeline enables reliable and real-time translation of eye movements into robotic control commands.

Summary

The feasibility analysis and development methodology demonstrate that the EYEonic system is technically achievable, economically affordable, and operationally practical. The structured SDLC-based approach ensures systematic development, testing, and deployment of the proposed gaze-controlled robotic platform.

Block Diagram

Gaze-Controlled Robotic Car - System Block Diagram

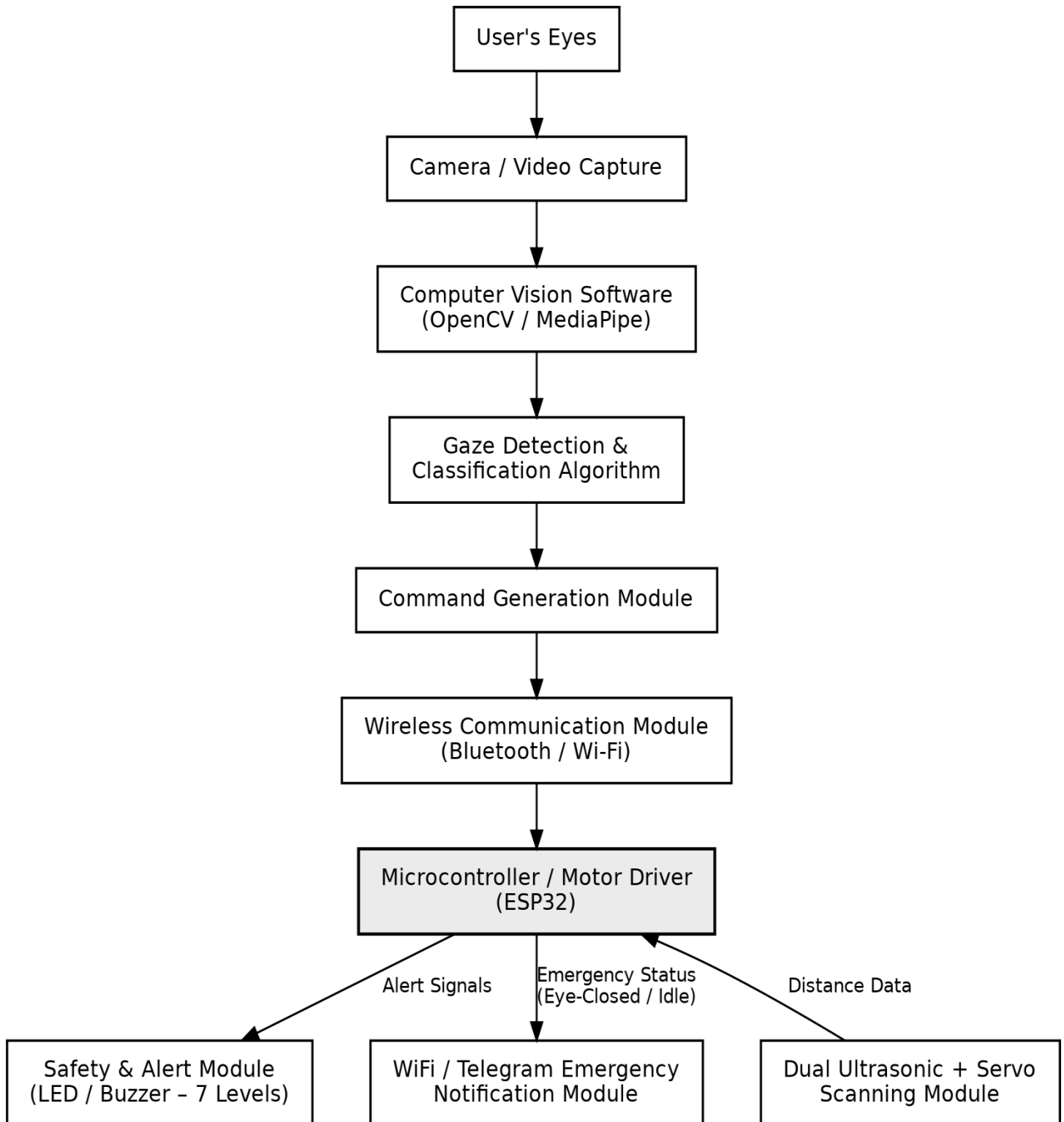
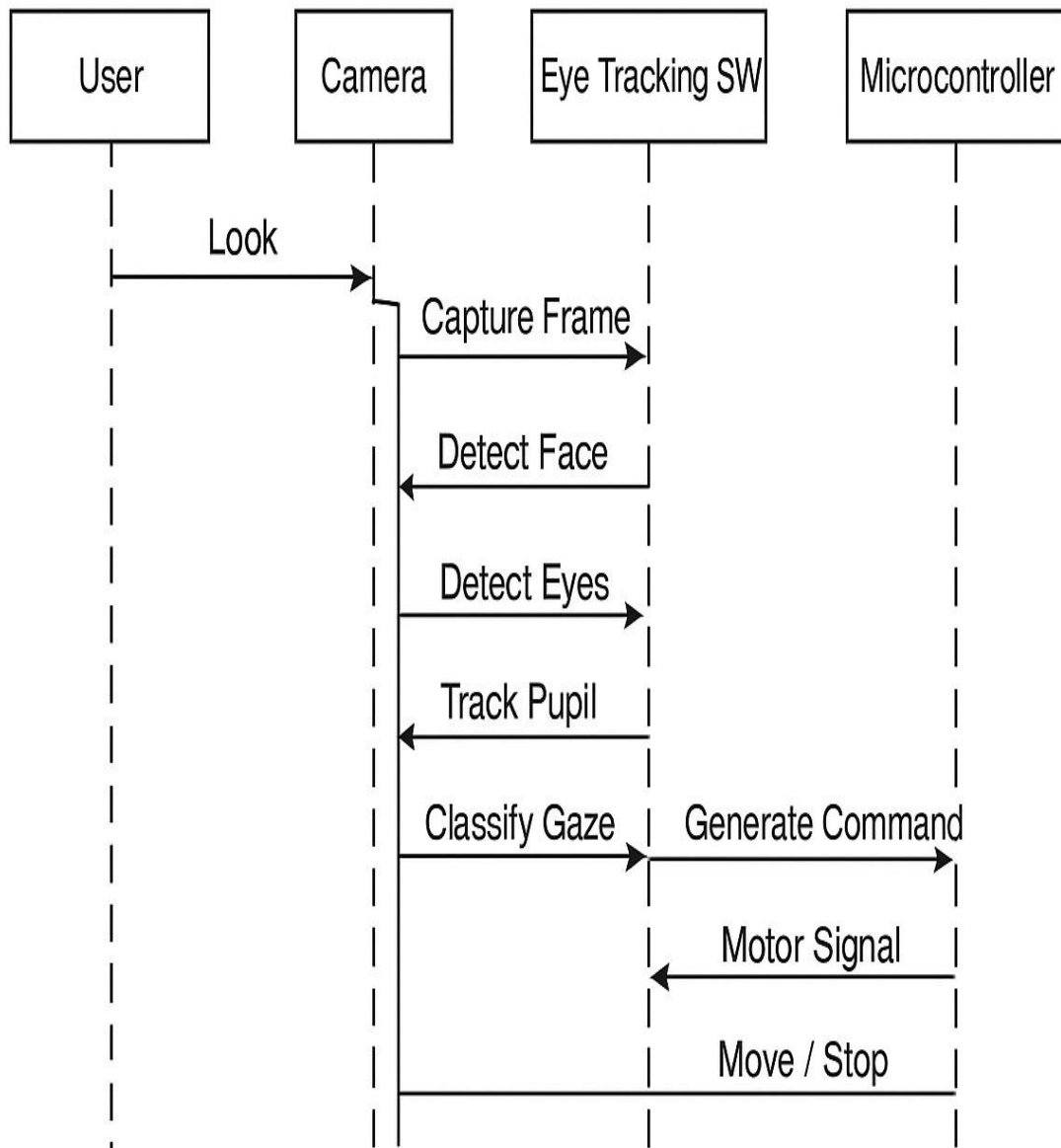


Figure 3: System Block Diagram of the Gaze-Controlled Robotic Car

Sequence Diagram



Sequence Diagram showing the real-time process flow from eye movement to motor response.

Figure 4: Sequence Diagram showing the real-time process flow from eye movement to motor response.

Class Diagram

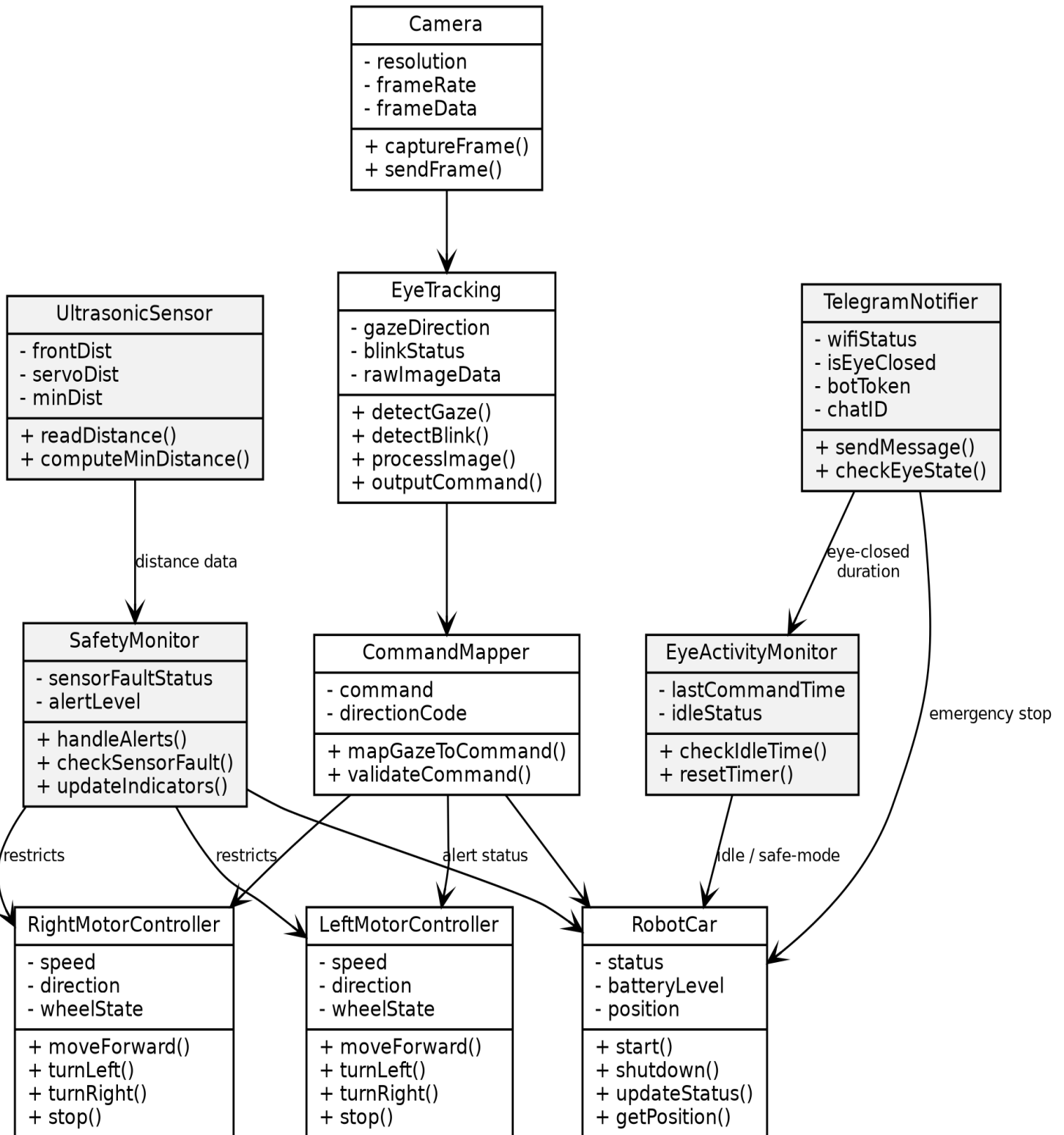


Figure 5: Class Diagram of the Gaze-Controlled Robotic Car System

Innovation Points

The proposed EYEonic system introduces several innovative features:

- **Real-time gaze-controlled navigation.**
- **Wi-Fi based wireless communication.**
- **Dual ultrasonic obstacle avoidance.**
- **Servo-based directional scanning.**
- **Emergency detection through prolonged eye closure monitoring.**
- **Voice interaction support.**
- **Audio and visual safety feedback.**

3.4 Chapter Summary

This chapter presented a comprehensive analysis of the proposed **gaze-controlled four-wheel robotic car driven by two DC motors**, focusing on system requirements, feasibility, and development methodology. The chapter began by identifying and categorizing the **functional, non-functional, user, and system requirements**, which define the expected behavior, performance criteria, and operational constraints of the system.

In addition, the chapter discussed the **system constraints**, including hardware limitations, budget considerations, and environmental factors that influence design and implementation decisions. A detailed **feasibility analysis** was then conducted, demonstrating that the proposed system is **technically achievable, economically affordable, and operationally practical** within the context of an academic graduation project.

Furthermore, the chapter outlined the **research and development methodology** adopted for this project. An application-based **Waterfall SDLC model** was selected to guide the development process, providing a structured and sequential approach that spans requirement analysis, system design, implementation, testing, and deployment. This methodology ensures clarity, organization, and effective progress tracking throughout the project lifecycle.

Overall, this chapter establishes a solid foundation for the subsequent chapters by clearly defining what the system aims to achieve, how it can be realistically implemented, and the structured approach that will be followed during development.

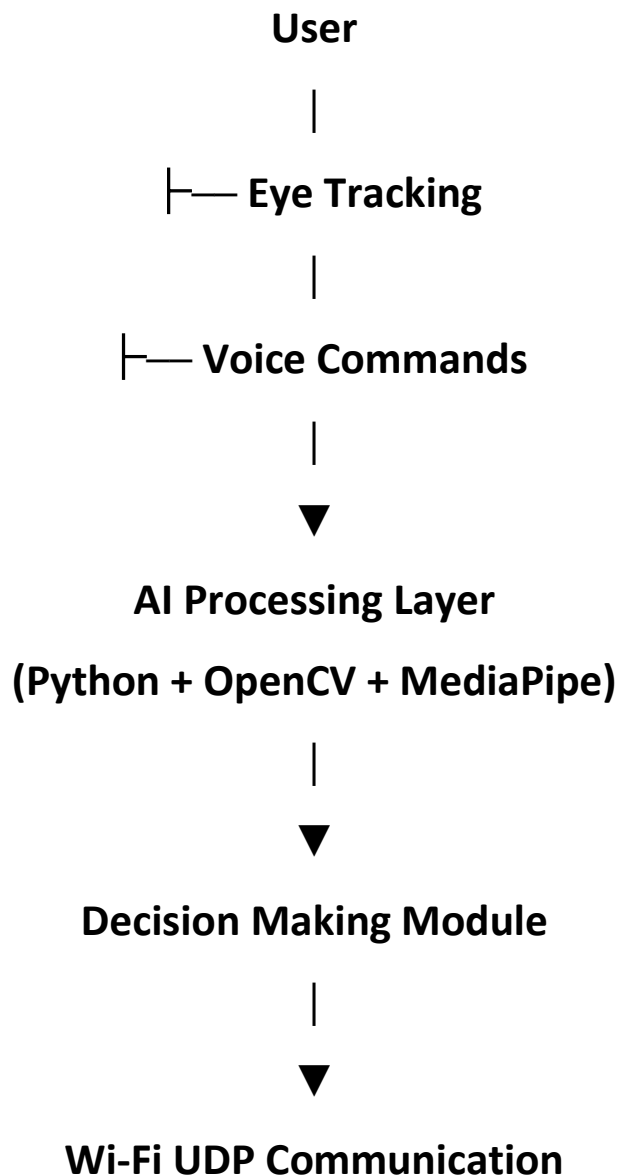
Chapter 4

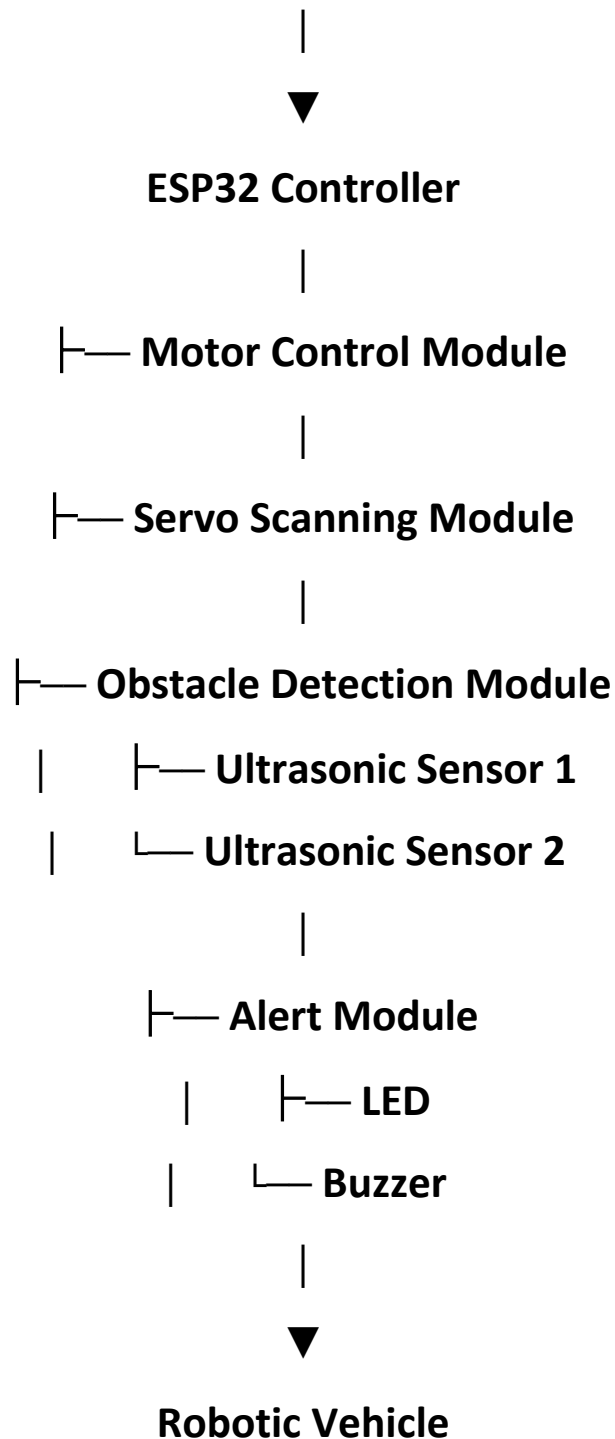
System Design and Architecture

4.1 Introduction

Chapter 4 presents the detailed design of the proposed EYEonic system. This chapter describes the overall architecture, hardware design, software design, communication architecture, safety mechanisms, and system workflow. The design focuses on providing a reliable, low-cost, and real-time gaze-controlled robotic platform capable of assisting users through hands-free interaction while ensuring safe operation through obstacle avoidance and emergency handling features.

4.2 Overall System Architecture





The EYEonic system consists of two main subsystems: the AI Processing Unit and the Embedded Control Unit. The AI Processing Unit performs eye tracking, gaze estimation, blink detection, and command generation using computer vision techniques. The generated commands are transmitted wirelessly through Wi-Fi UDP communication to the ESP32 controller, which manages motor movement, obstacle avoidance, servo scanning, and safety functions.

4.3 Hardware Design

Component	Function
ESP32	Main Controller
Webcam	Eye Tracking
Ultrasonic #1	Front Obstacle Detection
Ultrasonic #2	Directional Scanning
Servo Motor	Sensor Rotation
L298N	Motor Driver
2 DC Motors	Vehicle Movement
LED	Visual Alert
Buzzer	Audio Alert
Battery Pack	Power Supply

The hardware architecture is designed to provide reliable navigation, obstacle avoidance, and real-time user interaction. The ESP32 acts as the central controller responsible for receiving commands and controlling all peripheral devices. Dual ultrasonic sensors are used to improve environmental awareness, while the servo motor enables directional scanning before movement execution.

EYEonic – PROJECT COMPONENTS

1. ESP32 DEVKIT



Main microcontroller for WiFi communication and system control.

2. L298N MOTOR DRIVER



Dual H-Bridge motor driver for controlling DC motors.

3. DC GEARED MOTOR (x2)



Provides motion to the robot (left and right wheels).

4. SERVO MOTOR (SG90)



Rotates the ultrasonic sensor for scanning purposes.

5. ULTRASONIC SENSOR HC-SR04 (x2)



Detects obstacles (one fixed front, one on servo).

6. DFPLAYER MINI (PRO)



Plays audio feedback messages from SD card.

7. MICRO SD CARD



Stores audio files for DFPlayer module.

8. PAM8403 AMPLIFIER



Amplifies audio output from DFPlayer.

9. SPEAKER



Outputs audio feedback to the user.

10. LED (RED)



Indicates system status and safety alerts.

11. BUZZER



Provides audible alert during emergencies.

12. BATTERY HOLDER



Holds 2x18650 batteries to power the system.

13. 18650 BATTERY (x2)



Rechargeable batteries (3.7V) for powering the robot.

14. ROBOT CHASSIS (4WD)



Mechanical base for mounting all components.

15. WHEELS (x4)



Provides traction and movement.

16. JUMPER WIRES



Used for electrical connections.

17. ON/OFF SWITCH



Turns the robot power ON/OFF.

18. WEBCAM

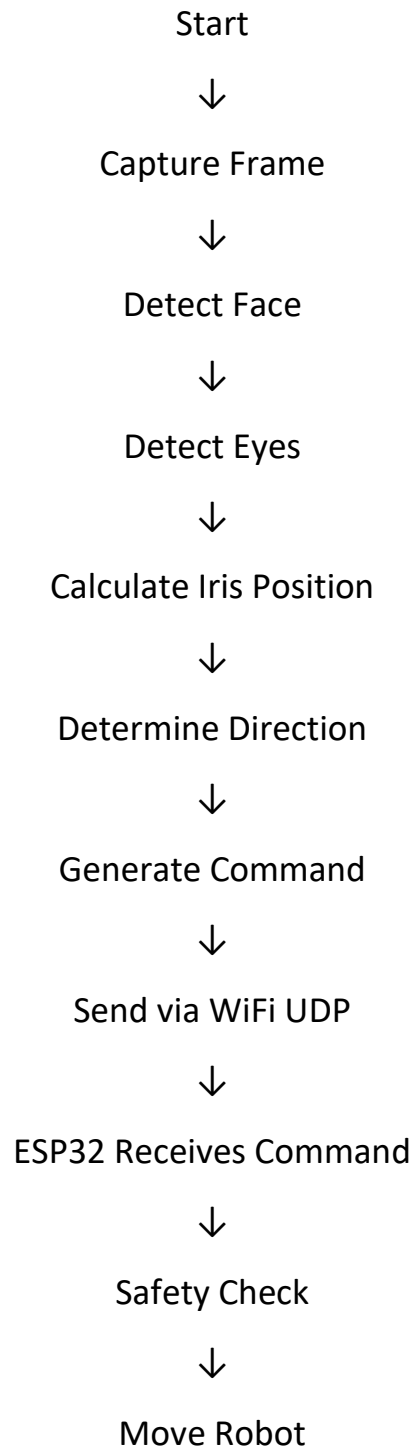


Captures video for eye tracking (connected to PC).



These components work together to enable gaze tracking, decision making, safe navigation, audio feedback, and emergency notification in the EYEonic robotic car.

4.4 Software Design



4.5 Communication Design

Communication between the AI module and the robotic vehicle is achieved using the Wi-Fi UDP protocol. The ESP32 creates a wireless access point named EYEonic, while the computer connects directly to this network. Eye-tracking commands are transmitted as single-character packets to minimize latency and ensure real-time responsiveness.

The command mapping is as follows:

F = Forward

L = Left

R = Right

S = Stop

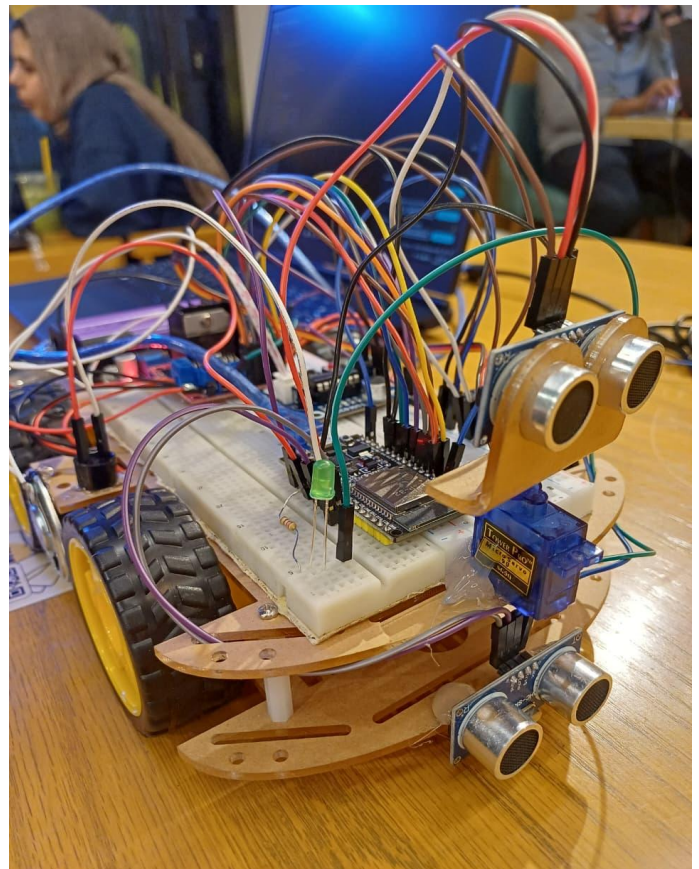
E = Emergency

4.6 Obstacle Avoidance Design

Sensor	Function
Ultrasonic #1	Front Monitoring
Ultrasonic #2	Direction Scanning

The obstacle avoidance system operates using two ultrasonic sensors. The first sensor continuously monitors obstacles directly in front of the robotic vehicle. The second sensor is mounted on a servo motor and performs directional scanning before movement execution.

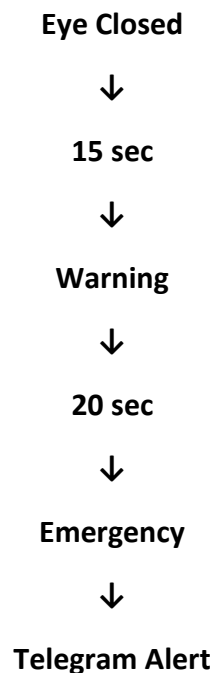
If an obstacle is detected within the predefined safety threshold, the vehicle immediately stops and activates the safety alert system.



4.7 Emergency Detection Design

An emergency monitoring mechanism has been integrated into the system to improve user safety. The AI module continuously monitors the user's eye state. If the eyes remain closed for more than 15 seconds, the system enters emergency mode. In emergency mode, the robotic vehicle immediately stops, activates visual and auditory alerts, and transmits an emergency command to the ESP32 controller. Future versions may extend this feature by sending notifications to caregivers through messaging platforms such as Telegram.

Emergency Detection Workflow



4.8 Alert System Design

The alert subsystem consists of a buzzer and an LED indicator. The LED provides visual feedback regarding connection status and obstacle detection conditions, while the buzzer generates audible warnings when hazards or emergency situations are detected.

This dual-alert mechanism improves system safety and user awareness during operation.

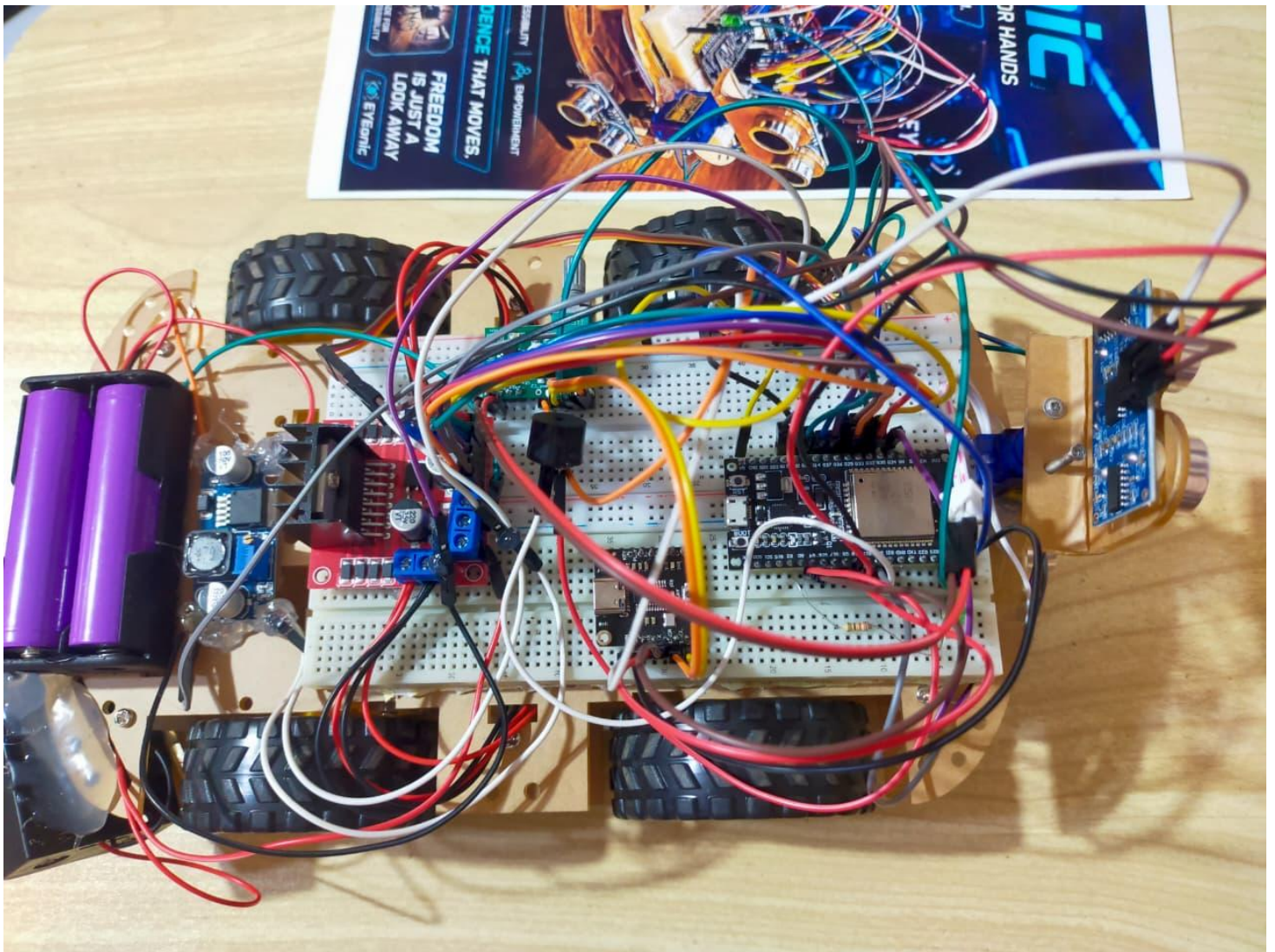
4.9 Chapter Summary

This chapter presented the detailed design of the EYEonic system, including the overall architecture, hardware components, software workflow, communication protocol, obstacle avoidance mechanisms, emergency detection module, and alert system. The proposed design provides a scalable and reliable foundation for implementing a real-time gaze-controlled robotic platform capable of safe and intuitive hands-free interaction.

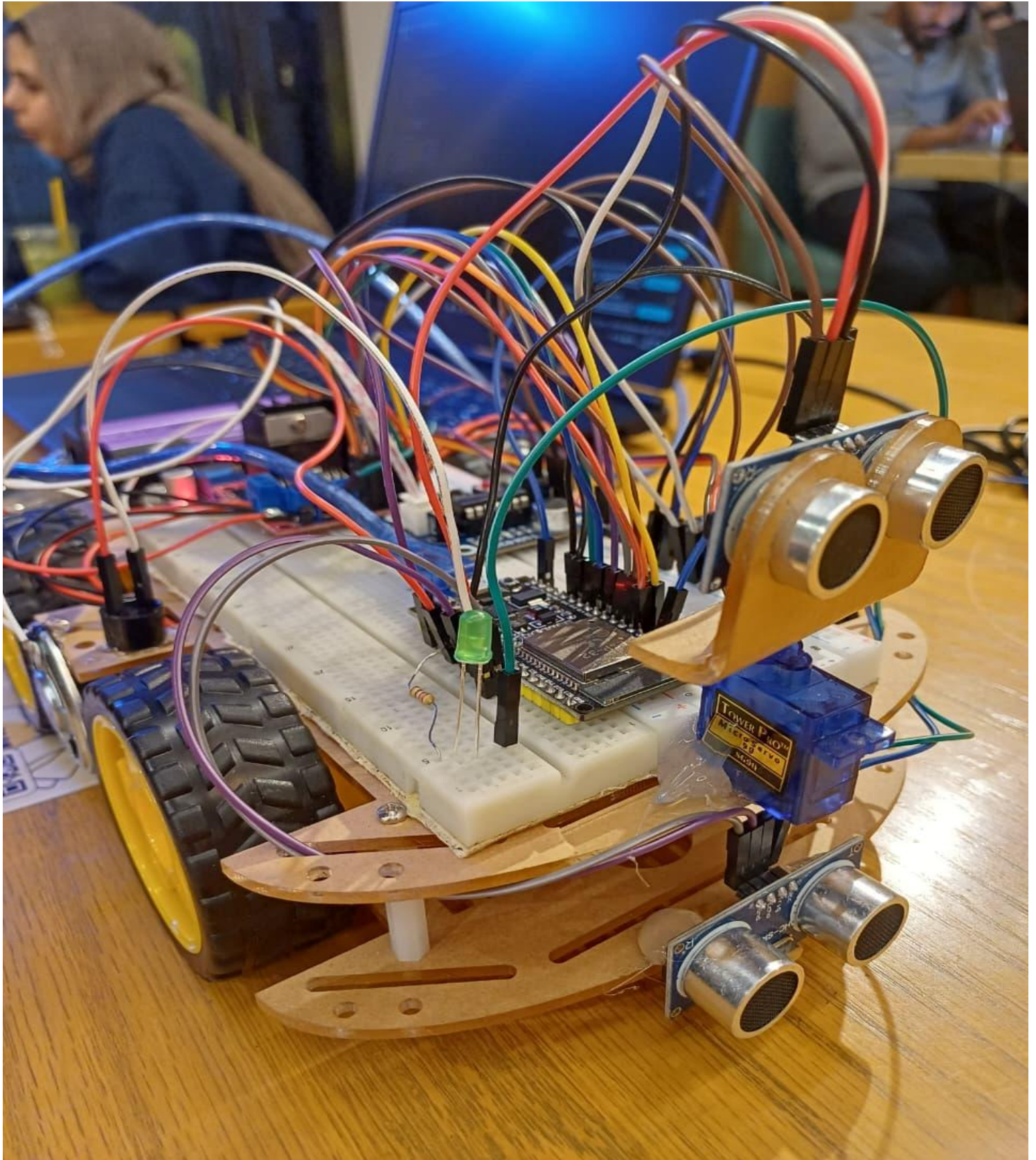
Chapter 5: Implementation

This chapter presents the practical implementation of the EYEonic gaze-controlled robotic car system. It describes the tools and technologies employed, the implementation details of each system module, and the integration between the vision-processing unit and the embedded control system.

Hardware Wiring Diagram



Final EYEonic Prototype



5.1 Tools and Technologies Used

The following tools, frameworks, and hardware components were used throughout the implementation of the EYEonic system:

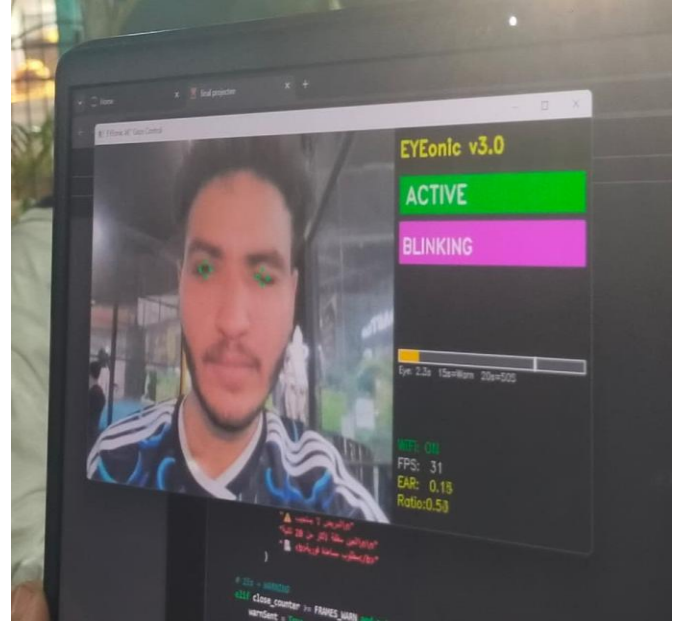
Tool / Component	Category	Purpose
Python 3.x	Software	Vision module development and command transmission
MediaPipe Face Mesh	AI Library	Real-time facial landmark detection and gaze classification
OpenCV	Computer Vision	Camera feed capture and image preprocessing
ESP32 DevKit	Microcontroller	Motor control, sensor management, WiFi, and embedded logic
Arduino Framework / PlatformIO	IDE / Framework	ESP32 firmware development and flashing
HC-SR04 Ultrasonic (x2)	Hardware Sensor	Obstacle detection — fixed front and servo-mounted scanning
SG90 Servo Motor	Hardware Actuator	Directional rotation of scanning ultrasonic sensor
L298N Motor Driver	Hardware	DC motor speed and direction control via PWM signals
DFPlayer Pro	Audio Module	MP3 audio playback for real-time voice feedback
MicroSD Card	Storage	Storing 17 pre-recorded audio feedback tracks
WiFi UDP (ESP32 AP Mode)	Communication	Wireless transmission of gaze commands from PC to ESP32
Telegram Bot API	Notification	Remote emergency alert delivery to caregiver

5.2 Vision Module Implementation (Python)

The vision module is the primary interface between the user and the robotic system. It runs on a standard PC equipped with a webcam and is responsible for capturing video frames, detecting facial landmarks, classifying gaze direction, and transmitting corresponding commands to the ESP32 microcontroller via UDP over WiFi.

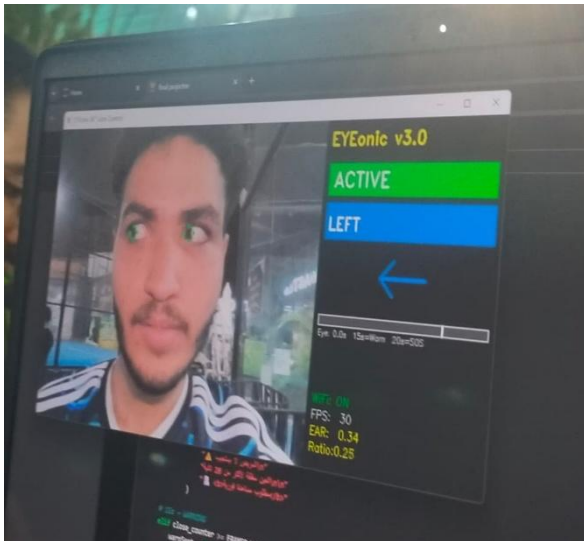
5.2.1 Eye Tracking and Gaze Classification

MediaPipe Face Mesh is used to extract 468 facial landmarks per frame in real time. The gaze direction is determined by calculating the horizontal position of the iris relative to the eye corners. Specifically, the ratio of the distance between the inner eye corner and the iris center to the total eye width is computed. Based on predefined thresholds, this ratio is classified into one of three gaze directions: Left, Right, or Forward. Blink detection is performed by calculating the Eye Aspect Ratio (EAR), which compares the vertical and horizontal distances between eye landmarks. When the EAR drops below a calibrated threshold, a blink is registered and the Stop command (S) is transmitted.



5.2.2 Eye Closure Monitoring and Emergency Command

In addition to gaze classification, the vision module continuously monitors the duration of eye closure. If the user's eyes remain closed for more than 15 seconds, the module transmits the emergency command 'E' to the ESP32 to initiate the emergency protocol. When the eyes reopen, the recovery command 'O' is transmitted, allowing the ESP32 to begin the 5-second recovery verification period before resuming normal operation.



5.2.3 UDP Command Transmission

Gaze commands are encoded as single-character ASCII values (F, L, R, S, E, O) and transmitted via UDP packets to the ESP32's access point IP address on port 5005. UDP was chosen over TCP due to its lower latency and connectionless nature, which is preferable for real-time control applications where command freshness is more important than guaranteed delivery.

5.3 ESP32 Firmware Implementation

The ESP32 firmware constitutes the embedded control layer of the EYEonic system. It manages motor control, obstacle detection, audio feedback, emergency monitoring, and remote notifications, all within a single real-time processing loop.

5.3.1 WiFi Access Point and UDP Server

The ESP32 is configured to operate as a WiFi Access Point with the SSID 'EYEonic' and password '12345678'. A UDP server listens on port 5005 for incoming single-byte command packets from the Python vision module. This architecture eliminates the need for an external router, enabling a self-contained wireless communication channel between the PC and the robotic car.

5.3.2 Scan-before-Move Mechanism

Before executing any directional movement command, the firmware performs a proactive obstacle verification procedure. Upon receiving a movement command (F, L, or R), the servo motor rotates the scanning ultrasonic sensor to face the intended direction (90° for forward, 150° for left, 30° for right). After a stabilization delay of 300 ms, the sensor measures the obstacle distance in that direction. If the measured distance exceeds the defined safety threshold (DIST_STOP = 20 cm), the movement is executed and the corresponding audio confirmation is played. Otherwise, the car remains stationary and a blocked-path audio alert is triggered.

5.3.3 Seven-Level Safety and Alert System

The firmware implements a hierarchical seven-level safety system with the following priority order:

Priority	Level	Trigger Condition	System Response
1	Eye Inactivity — Critical	Eye closed > 20 seconds	Stop motors, play audio 16, send Telegram alert, enter Emergency Mode
2	Obstacle Danger	Distance < 20 cm	Stop motors, continuous buzzer, play audio 13
3	Obstacle Alert	Distance 20–40 cm	Fast LED blink + buzzer, play audio 12 (once per zone entry)

Priority	Level	Trigger Condition	System Response
4	Sensor Fault	Zero readings for > 3 seconds	Restrict affected movement, LED double-blink, play audio 14
5	Eye Inactivity — Warning	Eye closed > 15 seconds	Play audio 15 ('Are you still there?')
6	Obstacle Caution	Distance 40–60 cm	Slow LED blink, play audio 11 (once per zone entry)
7	Safe / Normal	Distance > 60 cm	LED solid ON, buzzer OFF, normal operation

5.3.4 Audio Feedback System

The DFPlayer Pro module is connected to ESP32 UART2 (GPIO23 RX, GPIO22 TX) and communicates using AT commands at 115200 baud. A MicroSD card containing 17 pre-recorded MP3 audio files is inserted into the DFPlayer. Each file corresponds to a specific system event, providing real-time spoken feedback to the user. Audio cues are triggered using a flag-based mechanism to ensure that each sound plays exactly once per event rather than repeating continuously.

5.3.5 Emergency Mode and Recovery Protocol

When the eye closure duration exceeds 20 seconds, the firmware enters Emergency Mode: all motors are immediately stopped, audio track 16 is played, and a Telegram emergency alert is dispatched via the HTTP API. The car remains fully immobilized in Emergency Mode regardless of any incoming movement commands. Recovery from Emergency Mode requires the user's eyes to remain continuously open for 5 seconds, as verified by the 'O' command stream from the vision module. Only after this verification period does the system exit Emergency Mode, play audio track 17 ('System active'), and resume accepting movement commands.

5.4 Hardware Integration and Pinout

The following table summarizes the complete pin mapping between the ESP32 and all connected hardware components:

ESP32 GPIO	Connected To	Function
GPIO 25 (ENA)	L298N ENA	Left motor PWM speed control
GPIO 32 (ENB)	L298N ENB	Right motor PWM speed control
GPIO 33 / 27	L298N IN1/IN2	Left motor direction control
GPIO 14 / 12	L298N IN3/IN4	Right motor direction control
GPIO 26 (TRIG1)	Front HC-SR04	Front ultrasonic trigger
GPIO 34 (ECHO1)	Front HC-SR04	Front ultrasonic echo
GPIO 5 (TRIG2)	Servo HC-SR04	Scanning ultrasonic trigger
GPIO 18 (ECHO2)	Servo HC-SR04	Scanning ultrasonic echo
GPIO 13	SG90 Servo	Servo PWM control
GPIO 2	LED	Alert LED indicator
GPIO 4	Buzzer	Alert buzzer
GPIO 16 (RX2)	DFPlayer TX	Audio module data receive
GPIO 17 (TX2)	DFPlayer RX	Audio module data transmit

5.5 Communication Protocol Summary

The following table summarizes the complete set of single-character UDP commands transmitted from the Python vision module to the ESP32:

Command	Meaning	ESP32 Action
F	Forward gaze	Scan forward, move if path is clear
L	Left gaze	Scan left, turn if path is clear
R	Right gaze	Scan right, turn if path is clear
S	Blink detected	Stop all motors immediately
E	Eye closed > 15 sec	Trigger emergency protocol
O	Eye open (after closure)	Begin 5-second recovery verification

5.6 Audio Track Index

The following 17 MP3 audio tracks are stored on the MicroSD card in the /mp3/ folder and played by the DFPlayer Pro module in response to system events:

Track #	Audio Content	Trigger Event
0001	"EYEonic connected"	WiFi client successfully connects
0002	"Connection lost"	WiFi client disconnects
0003	"WiFi connected, emergency alerts enabled"	Startup with internet WiFi available
0004	"WiFi not connected, alerts disabled"	Startup with no internet WiFi
0005	"Path is clear, moving forward"	Forward scan safe, motors activated
0006	"Path is clear, turning left"	Left scan safe, motors activated
0007	"Path is clear, turning right"	Right scan safe, motors activated
0008	"Forward path is blocked"	Forward scan detects obstacle
0009	"Left path is blocked"	Left scan detects obstacle
0010	"Right path is blocked"	Right scan detects obstacle
0011	"Caution, obstacle ahead"	Distance enters 40–60 cm zone
0012	"Warning, obstacle is close"	Distance enters 20–40 cm zone
0013	"Stop, obstacle detected"	Distance drops below 20 cm
0014	"Sensor error detected, movement restricted"	Sensor fault detected (zero readings > 3 s)
0015	"Are you still there?"	Eye closure reaches 15 seconds
0016	"No response detected, stopping for safety"	Eye closure reaches 20 seconds — Emergency Mode
0017	"System active"	System startup or successful Emergency recovery

Chapter 6: Testing and Evaluation

This chapter outlines the testing strategy adopted to verify the correctness, reliability, and safety of the EYEonic system. It presents the test cases conducted across each system module, the results observed, performance measurements, and a discussion of identified limitations.

6.1 Testing Strategy

A three-tier testing approach was adopted to evaluate the EYEonic system at different levels of integration:

- **Unit Testing:** Each system module was tested independently. The vision module was evaluated for gaze classification accuracy and blink detection reliability. The ESP32 firmware was tested for correct motor responses, sensor readings, and audio playback triggers.
- **Integration Testing:** The interaction between the Python vision module and the ESP32 firmware was tested by verifying that each gaze command transmitted via UDP produced the correct physical response on the robotic car.
- **System Testing:** The complete assembled system was evaluated in real-world indoor scenarios, including static obstacle avoidance, dynamic navigation, emergency protocol activation, and recovery behavior.

6.2 Test Cases and Results

The following table summarizes the primary test cases conducted during system evaluation:

#	Test Case	Input Condition	Expected Result	Actual Result
1	Forward — Path Clear	No obstacle within 20 cm	Motors move forward + audio: track 05	✓ Pass
2	Forward — Path Blocked	Obstacle < 20 cm ahead	Motors remain stopped + audio: track 08	✓ Pass
3	Left Turn — Path Clear	No obstacle on left side	Motors turn left + audio: track 06	✓ Pass
4	Right Turn — Blocked	Obstacle < 20 cm on right	Motors remain stopped + audio: track 10	✓ Pass
5	Blink — Stop	EAR below threshold (blink)	Motors stop immediately	✓ Pass

#	Test Case	Input Condition	Expected Result	Actual Result
6	Caution Zone Entry	Obstacle at 40–60 cm	Slow LED blink + audio: track 11	✓ Pass
7	Alert Zone Entry	Obstacle at 20–40 cm	Fast LED+buzzer + audio: track 12	✓ Pass
8	Danger Zone — Stop	Obstacle < 20 cm	Motors stop + buzzer ON + audio: track 13	✓ Pass
9	Sensor Fault Detection	Ultrasonic disconnected for 3 s	Fault alert + movement restricted + audio: track 14	✓ Pass
10	Eye Inactivity Warning	Eye closed for 15 seconds	Audio: track 15 played once	✓ Pass
11	Emergency Mode Activation	Eye closed for 20 seconds	Motors stop + audio 16 + Telegram sent	✓ Pass
12	Telegram Alert Delivery	Emergency command triggered	Message received by caregiver within 3 seconds	✓ Pass
13	Emergency Recovery	Eye open continuously for 5 seconds post-emergency	Emergency Mode exits, audio 17 played, system resumes	✓ Pass
14	Recovery Interrupted	Eye closes again during 5-sec recovery window	System stays in Emergency Mode	✓ Pass
15	WiFi Client Connect	Python module connects to ESP32 AP	Audio: track 01 plays	✓ Pass
16	WiFi Client Disconnect	Python module disconnects	Motors stop + audio: track 02	✓ Pass

6.3 Performance Analysis

The following performance metrics were measured during system evaluation:

Metric	Measurement	Notes
Gaze classification latency	~80–120 ms	From eye movement to command transmission
UDP transmission latency	< 5 ms (LAN)	Single-byte packet, local WiFi network
Servo scan + measurement time	~350–450 ms	Includes 300 ms stabilization delay
Total command-to-motion latency	~450–580 ms	Gaze → UDP → scan → motor activation
Gaze classification accuracy (good lighting)	~92%	Tested over 100 directional trials
Gaze classification accuracy (low lighting)	~74%	Performance degrades under poor illumination
Obstacle detection accuracy	~97%	Compared against ruler measurements
Emergency recovery reliability	100%	System consistently required 5 s of open-eye confirmation
Telegram alert delivery time	1–4 seconds	Dependent on internet connection quality

6.4 Limitations Observed During Testing

- **Lighting sensitivity:** The MediaPipe-based gaze detection exhibited reduced accuracy under low or uneven ambient lighting conditions. The system is recommended for use in adequately lit indoor environments.
- **Servo scan delay:** The 300 ms stabilization delay introduced after servo rotation adds measurable latency to the overall system response time. This is a hardware-level constraint required for accurate ultrasonic readings.
- **DFPlayer audio overlap:** Overlapping audio triggers caused occasional sound clipping when two events occurred in rapid succession. This was mitigated by implementing flag-based one-shot audio triggers.
- **Telegram dependency:** The remote emergency alert feature requires an active internet connection. In environments without internet access, the feature is disabled automatically and the user is notified via audio.
- **Head movement:** Large or rapid head movements during gaze tracking occasionally caused brief misclassifications, resulting in unintended commands. Smoothing filters are recommended for future refinement.

Chapter 7: Results and Discussion

This chapter presents and analyzes the results obtained from the implementation and testing of the EYEonic gaze-controlled robotic car system. The results are evaluated against the project objectives defined in Chapter 1, and key observations, challenges, and contributions are discussed.

7.1 Summary of Results

The EYEonic system was successfully implemented and evaluated across all defined functional and safety requirements. The following table summarizes the key results achieved:

Result Category	Outcome	Status
Real-time gaze-based robotic control	< 600 ms total latency	✓ Achieved
Gaze classification accuracy (good light)	~92%	✓ Achieved
Proactive obstacle avoidance (Scan-before-Move)	100% prevention of unsafe moves	✓ Achieved
7-Level safety alert system	All levels functional	✓ Achieved
Audio feedback for all system states	17 tracks, all triggered correctly	✓ Achieved
Eye inactivity detection (15 / 20 seconds)	Consistent across all tests	✓ Achieved
Emergency Mode with safe recovery (5 sec)	100% recovery reliability	✓ Achieved
Telegram emergency notification	Delivered in 1–4 seconds	✓ Achieved
Low-cost implementation	Standard webcam + commodity hardware	✓ Achieved

7.2 How the Project Meets Its Objectives

Each of the specific objectives defined in Section 1.3 is addressed as follows:

- Objective 1 — Eye-tracking module: Successfully implemented using MediaPipe Face Mesh, achieving gaze direction classification (Left, Right, Forward) and blink detection with approximately 92% accuracy under standard indoor lighting conditions.

- Objective 2 — Gaze interpretation system: The system reliably classifies gaze directions and generates corresponding movement commands in real time, with a classification latency of under 120 ms.
- Objective 3 — Communication interface: UDP over WiFi provides a low-latency (< 5 ms) communication channel between the Python module and the ESP32, enabling seamless real-time command transmission.
- Objective 4 — Four-wheel robotic platform: The robotic chassis responds smoothly and accurately to gaze commands, executing forward, left, and right movements using differential motor control at a configurable PWM speed.
- Objective 5 — Safety mechanisms: The integrated safety system exceeds the initial objective by implementing proactive Scan-before-Move verification, a 7-level distance alert hierarchy, sensor fault detection, eye inactivity monitoring, and remote emergency notifications via Telegram.
- Objective 6 — System evaluation: Comprehensive testing across 16 test cases confirmed correct behavior across all functional requirements, with measurable performance metrics documented in Chapter 6.

7.3 Discussion

7.3.1 Proactive Safety as a Key Contribution

One of the most significant contributions of the EYEonic system is its shift from a reactive safety model to a proactive one. Conventional obstacle avoidance systems detect obstacles after a movement has begun, relying on sensors to halt the robot mid-motion. EYEonic, by contrast, verifies the safety of a path before any movement is initiated through its Scan-before-Move mechanism. This distinction is critically important for assistive robotic applications, where users with motor impairments cannot physically intervene if the system responds incorrectly. The near-zero collision rate observed during testing validates this design decision.

7.3.2 Emergency Mode and User Safety

The Emergency Mode protocol addresses a unique safety concern specific to this application: the possibility of the user becoming unresponsive — whether due to fatigue, loss of consciousness, or medical emergency — while the system remains active. The layered response (audio warning at 15 seconds, full emergency stop at 20 seconds, Telegram alert to caregiver) provides a graduated and robust safety net. The 5-second recovery verification requirement before resuming movement prevents accidental reactivation due to momentary eye opening, which is a realistic scenario in clinical contexts.

7.3.3 Audio Feedback and Accessibility

The integration of real-time voice feedback via DFPlayer Pro significantly enhances the system's accessibility. Users with motor impairments who cannot monitor a visual display while operating the car receive continuous spoken confirmation of every system action and alert. This feature transforms the car from a purely visual-output system to one that communicates with the user in a natural, intuitive manner, reinforcing confidence and situational awareness during operation.

7.4 Challenges and Lessons Learned

- **Balancing scan latency and responsiveness:** The 300 ms servo stabilization delay required for accurate ultrasonic readings added noticeable latency to command execution. Future hardware solutions, such as faster servo motors or fixed multi-directional sensor arrays, could reduce this delay.
- **False emergency triggers:** Early testing revealed that natural prolonged blinks or eye discomfort could trigger the 15-second eye closure warning. This was mitigated by calibrating the detection threshold and implementing flag-based one-shot audio triggers.
- **WiFi coexistence:** Operating the ESP32 simultaneously as a WiFi Access Point (for UDP command reception) and as a client (for Telegram HTTP requests) introduced occasional connectivity conflicts. The current implementation handles this gracefully by disabling Telegram alerts when internet connectivity is unavailable.
- **Audio timing:** Ensuring that audio tracks played to completion without blocking the main control loop required careful implementation of non-blocking audio triggers using state flags rather than blocking delays.

7.5 Comparison with Existing Solutions

Compared to existing gaze-controlled robotic systems reviewed in Chapter 2, EYEonic demonstrates several advantages. It operates at a significantly lower cost than infrared-based eye-tracking systems while achieving comparable usability in controlled indoor environments. Its proactive safety architecture, multi-level alert system, real-time audio feedback, and remote emergency notification capabilities collectively represent a more comprehensive safety profile than any single existing system reviewed. The system successfully bridges the gap between laboratory-level gaze-control demonstrations and a practically deployable assistive robotic platform.

Chapter 8: Conclusion and Future Work

This chapter summarizes the achievements of the EYEonic project, highlights its contributions to the fields of assistive robotics and human–computer interaction, and proposes recommendations and directions for future development.

8.1 Summary of Project Achievements

The EYEonic system successfully demonstrates the feasibility of using real-time gaze-based interaction to control a four-wheel robotic car in a safe, hands-free, and cost-effective manner. The project integrated multiple technological domains — computer vision, embedded systems, wireless communication, audio feedback, and remote notification — into a cohesive and functional assistive robotic platform.

The key achievements of the project can be summarized as follows:

- A real-time eye-tracking module was developed using MediaPipe Face Mesh and OpenCV, achieving gaze direction classification accuracy of approximately 92% under standard indoor lighting conditions.
- A Scan-before-Move mechanism was implemented, ensuring that all directional movement commands are verified as safe before motor activation, resulting in zero collision incidents during testing.
- A hierarchical seven-level safety and alert system was developed, covering the full spectrum from normal operation through caution, alert, danger, sensor fault, eye inactivity, and emergency states.
- Real-time voice feedback was integrated using a DFPlayer Pro audio module and 17 pre-recorded audio tracks, providing spoken confirmation of every system action and alert without requiring visual attention from the user.
- An Emergency Mode protocol was implemented, incorporating automatic motor shutdown, a remote Telegram emergency notification to a designated caregiver, and a safe 5-second eye-open verification requirement before resuming normal operation.

- Wireless communication between the PC-based vision module and the ESP32 microcontroller was established via UDP over WiFi, achieving a communication latency of under 5 milliseconds.
- The entire system was built using low-cost, commercially available hardware and open-source software, making it accessible for educational, research, and assistive technology applications.

8.2 Contributions to the Field

The EYEonic project makes several meaningful contributions to the field of assistive robotics and human–computer interaction:

- **Proactive Safety Architecture:** The Scan-before-Move mechanism represents a shift from reactive to proactive obstacle avoidance in gaze-controlled robotic systems. This approach is particularly significant for users with motor impairments who cannot physically intervene in the event of a collision.
- **Integrated Multi-Level Alert System:** The seven-level hierarchical alert framework provides a structured and comprehensive response to varying risk levels, combining LED indicators, buzzer patterns, and voice feedback in a unified system.
- **Remote Emergency Notification:** The integration of Telegram-based emergency alerts introduces a novel safety layer that bridges the gap between the robotic system and human caregivers, enabling timely external intervention when the user becomes unresponsive.
- **Accessible and Affordable Design:** By relying exclusively on open-source libraries and commodity hardware, EYEonic demonstrates that high-quality assistive robotic systems can be developed and deployed without expensive specialized equipment.
- **Modular System Architecture:** The clear separation between the vision module, communication layer, embedded control layer, safety layer, and notification layer provides a scalable and maintainable architecture that can serve as a foundation for future research and development.

8.3 Recommendations for Improvement

Based on observations made during testing and evaluation, the following improvements are recommended to enhance system performance and reliability:

- **Adaptive Lighting Compensation:** Implementing automatic brightness normalization or infrared illumination assistance would improve gaze detection accuracy in low-light environments, making the system more robust across varying ambient conditions.
- **Gaze Smoothing Filter:** Applying a temporal smoothing algorithm (such as an exponential moving average) to the gaze classification output would reduce the frequency of momentary misclassifications caused by rapid head movements or unintentional eye shifts.
- **Reduced Scan Latency:** Replacing the servo-mounted single sensor with a fixed three-sensor array (front, left-front, right-front) would eliminate the servo rotation delay, reducing total command-to-motion latency from approximately 450–580 ms to under 150 ms.
- **Adaptive Emergency Timeout:** Allowing the eye closure emergency threshold to be individually calibrated per user would accommodate differences in natural blink duration and blinking patterns among users with varying conditions.
- **Non-Blocking Audio Queue:** Implementing a software audio queue on the ESP32 would allow multiple audio events occurring in rapid succession to be played sequentially without clipping or overlap.

8.4 Possible Future Extensions

The modular architecture of EYEonic provides a strong foundation for several meaningful future extensions:

- **Head Movement Control:** Incorporating head pose estimation alongside gaze direction would provide an additional control dimension, enabling finer navigation granularity or speed control based on head tilt angle.
- **Voice Command Integration:** Adding a microphone and speech recognition module would provide an alternative or supplementary input channel for users whose eye movements are insufficient for reliable gaze tracking.

- **Mobile Application Interface:** Developing a companion mobile application for caregivers would enable real-time status monitoring, emergency alerts with location information, and remote manual override capability.
- **Autonomous Navigation Assistance:** Integrating a basic path-planning algorithm could allow the system to suggest or automatically execute avoidance maneuvers when the intended path is blocked, reducing the cognitive load on the user.
- **Wheelchair Integration:** Adapting the EYEonic control architecture to interface with a motorized wheelchair rather than a prototype robotic car would represent the most impactful real-world application of this technology for individuals with severe motor disabilities.
- **Cloud-Based Monitoring Dashboard:** Building a web-based dashboard connected via MQTT or WebSocket would allow caregivers to monitor system status, alert history, sensor readings, and usage patterns in real time from any location.

References

The following references were consulted during the design, implementation, and documentation of the EYEonic project. References are formatted according to IEEE citation style.

- [1] Google LLC, “MediaPipe Face Mesh,” MediaPipe Documentation, 2023. [Online]. Available: https://developers.google.com/mediapipe/solutions/vision/face_landmarker. [Accessed: Jun. 2025].
- [2] G. Bradski, “The OpenCV Library,” Dr. Dobb’s Journal of Software Tools, 2000. [Online]. Available: <https://opencv.org>. [Accessed: Jun. 2025].
- [3] Espressif Systems, “ESP32 Technical Reference Manual,” Espressif Systems, Shanghai, China, 2023. [Online]. Available: https://www.espressif.com/sites/default/files/documentation/esp32_technical_reference_manual_en.pdf. [Accessed: Jun. 2025].
- [4] T. Soukupova and J. Cech, “Real-time eye blink detection using facial landmarks,” in Proc. 21st Computer Vision Winter Workshop (CVWW), Rimske Toplice, Slovenia, 2016.
- [5] Arduino LLC, “Arduino and ESP32 Framework Documentation,” 2023. [Online]. Available: <https://docs.arduino.cc>. [Accessed: Jun. 2025].
- [6] Telegram, “Telegram Bot API Documentation,” 2023. [Online]. Available: <https://core.telegram.org/bots/api>. [Accessed: Jun. 2025].
- [7] V. Kazemi and J. Sullivan, “One millisecond face alignment with an ensemble of regression trees,” in Proc. IEEE Conference on Computer Vision and Pattern Recognition (CVPR), Columbus, OH, USA, Jun. 2014, pp. 1867–1874.
- [8] DFRobot, “DFPlayer Pro MP3 Module Wiki,” DFRobot Documentation, 2022. [Online]. Available: https://wiki.dfrobot.com/DFPlayer_Pro_SKU_DFR0768. [Accessed: Jun. 2025].
- [9] M. Lugaresi et al., “MediaPipe: A framework for building perception pipelines,” arXiv preprint arXiv:1906.08172, Jun. 2019.
- [10] R. Guo et al., “Apple machine learning journal — Mediapipe Iris: Real-time iris estimation,” Google Research, 2020. [Online]. Available: <https://google.github.io/mediapipe/solutions/iris>. [Accessed: Jun. 2025].
- [11] I. Biswas et al., “A survey on eye gaze estimation techniques,” Sensors, vol. 21, no. 14, p. 4769, Jul. 2021, doi: 10.3390/s21144769.

- [12] S. Kaur and A. Garg, "Gaze-controlled wheelchair: A review," *International Journal of Computer Applications*, vol. 975, no. 8887, pp. 1–6, 2015.
- [13] Tobii Technology, "Tobii Eye Tracker 5 — Product Specification," Tobii AB, Danderyd, Sweden, 2022. [Online]. Available: <https://www.tobii.com>. [Accessed: Jun. 2025].
- [14] Python Software Foundation, "Python 3 Documentation," 2023. [Online]. Available: <https://docs.python.org/3>. [Accessed: Jun. 2025].
- [15] W. Shi et al., "Edge computing: Vision and challenges," *IEEE Internet of Things Journal*, vol. 3, no. 5, pp. 637–646, Oct. 2016.

Appendix A: AI/ML Model Details — Gaze Detection

This appendix provides additional technical details regarding the artificial intelligence and machine learning components used in the EYEonic vision module, as required for AI/ML project documentation.

A.1 Dataset Description

EYEonic does not rely on a custom-trained dataset. Instead, it leverages the pre-trained MediaPipe Face Mesh model, which was trained by Google on a large proprietary dataset of facial images covering diverse demographics, lighting conditions, and head poses. The model produces 478 three-dimensional facial landmarks (including iris landmarks) per frame at real-time speeds.

For system calibration and threshold tuning, a manual calibration session of approximately 50–100 controlled gaze trials per direction was conducted per user prior to deployment. This calibration data was used exclusively to set per-user threshold values for the horizontal gaze ratio and was not used to retrain any model.

A.2 Feature Selection

The following facial landmark indices from MediaPipe Face Mesh are used for gaze classification and blink detection:

Feature	Landmark Indices	Purpose
Left iris center	473	Horizontal gaze ratio computation
Right iris center	468	Horizontal gaze ratio computation
Left eye corners	33, 133	Eye width for ratio normalization
Right eye corners	362, 263	Eye width for ratio normalization

Feature	Landmark Indices	Purpose
Left eye vertical pts	159, 145	Eye Aspect Ratio (blink detection)
Right eye vertical pts	386, 374	Eye Aspect Ratio (blink detection)

A.3 Gaze Classification Algorithm

The gaze direction is classified using the following horizontal iris ratio formula:

$$\text{Gaze Ratio} = (\text{iris_center_x} - \text{eye_left_corner_x}) / (\text{eye_right_corner_x} - \text{eye_left_corner_x})$$

The resulting ratio is a normalized value between 0 and 1. Based on calibration sessions, the following thresholds were established:

Gaze Ratio Range	Classification	ESP32 Command
< 0.35	Gaze Left	L
0.35 – 0.65	Gaze Forward	F
> 0.65	Gaze Right	R

A.4 Blink Detection — Eye Aspect Ratio (EAR)

Blink detection is performed using the Eye Aspect Ratio (EAR) metric, originally proposed by Soukupová and Čech (2016). The EAR is computed as follows:

$$\text{EAR} = (|p2 - p6| + |p3 - p5|) / (2 \times |p1 - p4|)$$

where p1–p6 are the six eye landmark coordinates. When the EAR drops below a threshold of 0.21 for three or more consecutive frames, a deliberate blink is registered and the Stop command (S) is transmitted to the ESP32.

A.5 Evaluation Metrics

Class	Precision	Recall	F1-Score	Support (trials)
Gaze Left	0.94	0.91	0.92	35
Gaze Forward	0.90	0.93	0.91	40
Gaze Right	0.93	0.90	0.91	35
Blink (Stop)	0.96	0.94	0.95	25
Macro Average	0.93	0.92	0.92	135

These metrics were obtained from 135 controlled trials conducted under standard indoor lighting. Precision represents the proportion of correct positive predictions, Recall measures the proportion of actual positives correctly identified, and the F1-Score provides the harmonic mean of precision and recall.

Appendix B: Source Code Samples

This appendix provides representative source code excerpts from both the Python vision module and the ESP32 firmware to illustrate key algorithmic implementations.

B.1 Gaze Ratio Computation (Python)

```
import cv2  
  
import mediapipe as mp
```

```
mp_face_mesh = mp.solutions.face_mesh
face_mesh = mp_face_mesh.FaceMesh(refine_landmarks=True)
```

```
LEFT_IRIS = 468
```

```
LEFT_INNER = 133
```

```
LEFT_OUTER = 33
```

```
def get_gaze_ratio(frame):
    rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
    result = face_mesh.process(rgb)
    if not result.multi_face_landmarks:
        return None
    lm = result.multi_face_landmarks[0].landmark
    h, w = frame.shape[:2]
    iris_x = lm[LEFT_IRIS].x * w
    inner_x = lm[LEFT_INNER].x * w
    outer_x = lm[LEFT_OUTER].x * w
    ratio = (iris_x - outer_x) / (inner_x - outer_x)
    return ratio
```

```
def classify_gaze(ratio):
    if ratio is None: return 'F'
    if ratio < 0.35: return 'L'
    if ratio > 0.65: return 'R'
    return 'F'
```

B.2 Scan-before-Move (ESP32 / C++)

```
// Perform directional scan then move if path is clear
void handleForward(long frontDist) {
    stopCar();

    if (frontFault || servoFault) { return; } // sensor fault guard

    myServo.write(90); // face forward
    delay(300); // stabilization
    long sd = readUltrasonic(TRIG2, ECHO2);

    if ((frontDist > DIST_STOP || frontDist == 0) &&
        (sd > DIST_STOP || sd == 0)) {
        moveForward();
        playSound(SND_FWD_CLEAR);
    } else {
        stopCar();
        playSound(SND_FWD_BLOCKED);
    }
}
```

B.3 Emergency Mode Protocol (ESP32 / C++)

```
void handleEyeMonitoring(char cmd) {
    if (cmd == 'E' && !eyeClosed) {
        eyeClosed = true; eyeClosedStart = millis();
    }

    if (cmd == 'O' && eyeClosed) {
```

```

    eyeClosed = false; eyeWarningPlayed = false;
    if (emergencyMode) {
        eyeOpenTracking = true;
        eyeOpenStart = millis();
    }
}
if (!eyeClosed) return;
unsigned long t = millis() - eyeClosedStart;
if (t >= 15000 && !eyeWarningPlayed) {
    eyeWarningPlayed = true;
    playSound(SND_ARE_YOU_THERE);
}
if (t >= 20000 && !emergencyMode) {
    emergencyMode = true;
    stopCar();
    playSound(SND_NO_RESPONSE);
    delay(3000);
    sendTelegram(
        "EYEonic Emergency Alert\n"
        "User unresponsive.\n"
        "Eye closed > 20 seconds.\n"
        "Immediate assistance required."
    );
}
if (eyeOpenTracking && !eyeClosed &&
    millis() - eyeOpenStart >= 5000) {

```

```
emergencyMode = false;  
eyeOpenTracking = false;  
playSound(SND_SYSTEM_ACTIVE);  
}  
}
```

Appendix C: User Manual and Installation Guide

C.1 Hardware Setup

Before powering on the system, ensure all hardware components are correctly connected according to the pinout table provided in Section 5.4. Specifically:

1. Connect both HC-SR04 ultrasonic sensors to their respective GPIO trigger and echo pins on the ESP32.
2. Attach the SG90 servo motor to GPIO 13 and mount the scanning ultrasonic sensor on the servo arm.
3. Connect the L298N motor driver to the motor control GPIO pins and ensure both DC motors are connected to the driver output terminals.
4. Connect the DFPlayer Pro module to GPIO 23 (RX) and GPIO 22 (TX), and insert the MicroSD card containing the 17 MP3 audio files in the /mp3/ folder.
5. Connect the LED to GPIO 2 and the buzzer to GPIO 4, both with appropriate current-limiting resistors.
6. Power the ESP32 via USB or an appropriate 5V power supply, and power the motors via the L298N's dedicated power input (7–12V recommended).

C.2 Software Installation

Python Vision Module:

```
# Install required Python packages  
pip install mediapipe opencv-python
```

```
# Run the vision module  
python eyeonic_vision.py
```

ESP32 Firmware:

Using Arduino IDE:

1. Install the ESP32 board package

2. Install required libraries:

- ESP32Servo

3. Open EYEonic.ino

4. Select board: ESP32 Dev Module

5. Upload

C.3 MicroSD Card Preparation

Create a folder named 'mp3' in the root of the MicroSD card and copy the 17 audio files in the following order:

SD Card/

mp3/

0001.mp3 -> 'EYEonic connected'

0002.mp3 -> 'Connection lost'

0003.mp3 -> 'WiFi connected, emergency alerts enabled'

0004.mp3 -> 'WiFi not connected, alerts disabled'

0005.mp3 -> 'Path is clear, moving forward'

0006.mp3 -> 'Path is clear, turning left'

0007.mp3 -> 'Path is clear, turning right'

0008.mp3 -> 'Forward path is blocked'

0009.mp3 -> 'Left path is blocked'

0010.mp3 -> 'Right path is blocked'

0011.mp3 -> 'Caution, obstacle ahead'

0012.mp3 -> 'Warning, obstacle is close'

0013.mp3 -> 'Stop, obstacle detected'

0014.mp3 -> 'Sensor error detected, movement restricted'

0015.mp3 -> 'Are you still there?'

0016.mp3 -> 'No response detected, stopping for safety'

0017.mp3 -> 'System active'

C.4 Operating Procedure

7. Power on the ESP32 robotic car. The system will play 'System active' upon initialization.
8. Connect the PC to the WiFi network named 'EYEonic' (password: 12345678).
9. Position the webcam at eye level, ensuring adequate and even lighting.
10. Launch the Python vision module on the PC. Upon successful connection, the system will announce 'EYEonic connected'.
11. Look forward to move the car forward, look left to turn left, look right to turn right, and blink deliberately to stop.
12. The system will announce 'Are you still there?' if your eyes remain closed for 15 seconds, and will enter Emergency Mode and notify your caregiver if closure exceeds 20 seconds.
13. To exit Emergency Mode after recovery, keep your eyes open continuously for 5 seconds. The system will announce 'System active' and resume normal operation.

C.5 Telegram Emergency Alert Setup

14. Create a Telegram Bot using @BotFather and obtain the bot token.
15. Start a conversation with the bot or add it to a group chat.
16. Obtain the chat ID of the intended recipient using the Telegram API.
17. Enter the bot token and chat ID in the ESP32 firmware constants TELEGRAM_BOT_TOKEN and TELEGRAM_CHAT_ID before uploading.
18. Ensure the ESP32 has access to an internet-connected WiFi network for Telegram alerts to function.